A Survey on Binary and Ternary Neural Networks and Their Realization in Compute-in-Memory for Edge Intelligence

Dahoon Park, Hyungdong Park, Inguk Yeo, Jiyun Kim, Sunghyun Lee, Suhak Lee, Hyunseob Shin, *Student Member, IEEE*, Sung-il Pae, *Member, IEEE*, Deliang Fan, *Senior Member, IEEE*, Jaeha Kung, *Senior Member, IEEE*, and Kon-Woo Kwon, *Member, IEEE*

***Abstract*—Deep learning has achieved remarkable success across a wide range of applications, such as language modeling, computer vision, recommendation systems, and robotics. However, the growing size of models and their increasing computational demands pose significant challenges, particularly for resource-constrained devices. A promising approach to address these challenges is extreme quantization, exemplified by binary and ternary neural networks. These techniques significantly reduce model size by quantizing weights and activations to 1 or 1.58 bit, while simplifying computation, making them well-suited for efficient deployment in resource-limited environments. This article presents a comprehensive review of extreme quantization techniques, organized into three key areas: 1) a comparative analysis of quantizing only the weights [e.g., binary weight networks (BWNs) and ternary weight networks (TWNs)] versus quantizing both weights and activations (e.g., binary neural networks and ternary neural networks), along with a discussion of the progress and tradeoffs of their approaches; 2) an examination of how extreme quantization, initially applied to convolutional neural networks (CNNs), has been extended to Transformer architectures; and 3) an overview of compute-in-memory architectures optimized for binarization and ternarization, including designs based on advanced bit-cell technologies.**

***Index Terms*—Binary neural network, compute-in-memory, deep learning, quantization, ternary neural network (TNN).**

Received 25 September 2025; revised 8 November 2025; accepted 12 November 2025. Date of publication 17 November 2025; date of current version 8 January 2026. This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) under Grant RS-2022-II221170 and Grant RS-2023-0022984; in part by the National Research Foundation of Korea (NRF) funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2023-NR077140; in part by the BrainLink Program funded by the MSIT through the NRF under Grant RS-2023-00237308; in part by the 2025 Hongik University Innovation Support Program Fund; and in part by the "Regional Innovation System and Education (RISE)" through the Seoul RISE Center, funded by the Ministry of Education (MOE) and Seoul Metropolitan Government, under Grant 2025-RISE-01-035-05. *(Corresponding authors: Jaeha Kung; Kon-Woo Kwon.)*

Dahoon Park, Hyunseob Shin, and Jaeha Kung are with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea (e-mail: manyteacher93@korea.ac.kr; syncher99@korea.ac.kr; jhkung@korea.ac.kr).

Hyungdong Park, Inguk Yeo, Jiyun Kim, Sunghyun Lee, Suhak Lee, Sung-il Pae, and Kon-Woo Kwon are with the Department of Computer Engineering, Hongik University, Seoul 04066, South Korea (e-mail: parkrokr@g.hongik.ac.kr; yik6465@g.hongik.ac.kr; jiy00n1yun@gmail.com; dad0401@g.hongik.ac.kr; shsh7177@g.hongik.ac.kr; pae@hongik.ac.kr; konwoo@hongik.ac.kr).

Deliang Fan is with the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ 85281 USA (e-mail: dfan@asu.edu).

Digital Object Identifier 10.1109/JIOT.2025.3633487

I. Introduction

With the surge of big data, sophisticated algorithms, and hardware (HW) optimization, deep learning has become one of the powerful tools for daily applications. Earlier deep learning models have been designed to solve targeted applications, such as vision-related tasks using convolutional neural networks (CNNs) and language-related tasks using recurrent neural networks (RNNs). Recently, the development of deep learning models has evolved into much larger foundation models, e.g., Transformer-based models, capable of addressing a wide range of problems. However, the model size and the computational demand have grown rapidly, posing challenges for deployment. Thus, techniques such as quantization, network pruning, neural architecture search, and knowledge distillation to deploy these large models on the edge/Internet of Things (IoT) devices have become essential to significantly reduce the model size and the number of computations.

Among various techniques, quantization has become one of the most popular methods, which improves efficiency by reducing the bit-width of operands from single precision [32-bit floating point (FP32)] to 8-bit integer (INT8) or FP8, leading to a 4× reduction in memory requirement. In addition, INT8 multiply operations offer approximately 16× greater energy efficiency compared to FP32 multiply operations [1]. Such quantization techniques are applied to deep learning models through various optimization methods, such as quantization-aware training (QAT) and posttraining quantization (PTQ). However, in environments like microcontroller units (MCUs), where the available space for storing model parameters is extremely limited, quantization still faces significant challenges. For instance, even when a model like ResNet-18 [2], which has a total parameter size of 44.8 MB, is quantized to INT8, it is still impossible to deploy it on devices like the STM32U599 (which has <2.5-MB static random access memory (SRAM) and <4-MB flash memory). Thus, there is a growing need to explore more extreme forms of quantization.

Binarization (0/1) and ternarization (−1/0/+ 1) are extreme quantization techniques that drastically reduce bit-width by encoding values traditionally stored in the FP32 format as two states (1 bit) or three states ($\log_2 3 = 1.58$ bit), respectively.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

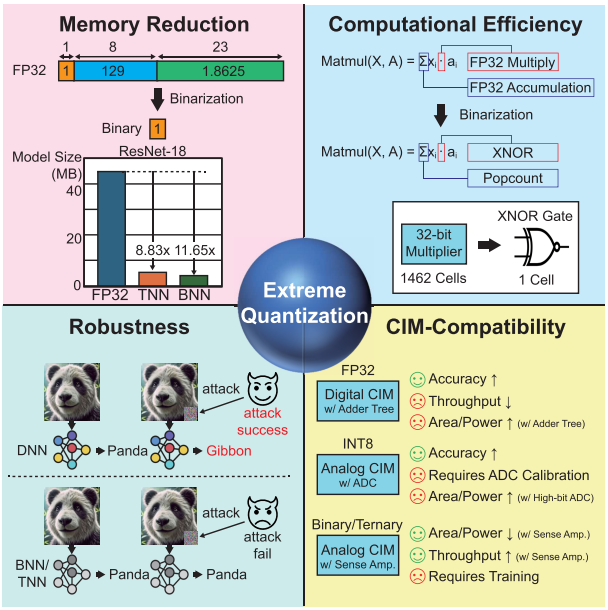


Fig. 1. Overview of extreme quantization advantages.

While these methods can result in a drop in accuracy due to quantization error, they provide many advantages: significant memory reduction, improved computational efficiency, enhanced robustness against adversarial attacks, and compatibility with compute-in-memory (CIM) architectures for ultralow power and area efficiency (see Fig. 1). For instance, binarizing ResNet-18 can reduce its model size to under 4 MB [3], addressing memory constraints in resource-limited devices like the STM32U599. Additionally, the reduced precision simplifies arithmetic operations, enabling faster and more energy-efficient computations on specialized hardware [4], [5]. The inherent noise tolerance in binary and ternary networks also makes them more resilient to decision-based closed-box adversarial attacks [6] and adversarial weight attacks [7], [8]. Moreover, their simplified logic is well-suited for CIM architectures, which integrate storage and computation for energy-efficient AI deployments on the embedded/IoT devices [9], [10], [11], [12]. These advantages drive the growing interest in extreme quantization for edge applications, motivating further exploration of their tradeoffs and potentials. In this work, we aim to conduct a comprehensive analysis of these two extreme quantization techniques to explore their effectiveness and tradeoffs on various applications.

This article distinguishes itself from other recent survey literature on binary neural networks (BNNs) [13], [14] in several key aspects. First, unlike prior work that focuses solely on reviewing binary networks, this article also provides a comprehensive review of ternary networks (Section III). By addressing both binarization and ternarization, this article broadens the spectrum of ultralow-bit quantization techniques, offering more flexible options for sub-2-bit quantization. Second, in this article, we evaluate the quality of binarization and ternarization on recent Transformer models for both vision and language tasks (Section IV). Although Transformers have become widely used not only in natural language processing [15], [16], [17], but also in computer vision [18], [19], [20], speech processing [21], and other fields [22], [23], [24], research on the application of binarization and ternarization to Transformers is still in an early stage. This article summarizes the existing methods for binarizing and ternarizing Transformer models such as BERT [15] and Vision Transformer (ViT) [18], providing valuable insights to guide further advancements in this area. Third, we examine kernel optimizations and software (SW) libraries designed to accelerate BNNs/ternary neural networks (TNNs) on the IoT and edge devices in Section V. We begin by addressing the inherent challenges of accelerating these models on off-the-shelf hardware and introduce bit packing as a fundamental solution. Following this, we demonstrate the extent of performance gains achievable through kernel-level acceleration that leverages bit packing on typical edge devices, including MCUs and Raspberry Pi. Then, we survey the landscape of existing libraries that support these models on commercial hardware, with the goal of improving the accessibility of ultralow-bit quantization.

In addition to the algorithmic innovation for edge intelligence, we review CIM hardware tailored to run BNNs and TNNs in Section VI. Although CIM has been extensively explored as a solution to the well-known memory wall problem using various memory devices and data mapping schemes, this article focuses on CIM architectures specifically optimized for BNNs and TNNs. By analyzing and comparing various CIM architectures designed for extremely low-power environments, this work aims to provide a broader range of solutions for applications where minimizing energy consumption is crucial. We further discuss the inherent limitation of standalone CIM architectures—their insufficiency in supporting the full range of operations required for complete, end-to-end AI applications. To overcome this, Section VII reviews the growing trend of integrating CIM with versatile RISC-5 processors. This section explores how this co-design strategy creates more capable deep learning accelerators, highlighting a key future direction for CIM in the IoT and edge devices.

II. Preliminaries

In neural networks, most of their runtime is consumed by matrix multiplications involved in projecting intermediate features to a different embedding space. The matrix multiplication can be expressed as

$$Y = I \odot W \tag{1}$$

where I and W represent the full-precision activations and weights, respectively, $\odot$ denotes the matrix multiplication, and Y represents the corresponding outputs. In full-precision neural networks, activations and weights are typically represented using floating-point values, meaning that the matrix multiplication relies on floating-point multipliers and adders [Fig. 2(a)], which introduce significant computational overheads. Moreover, storing weights as high-precision floating-point values leads to substantial energy consumption, particularly during data transfer between off-chip and on-chip memory [25]. To mitigate these challenges, we introduce four existing approaches that represent weights, or both activations and weights, in an extremely low bit-width, i.e., binarization or ternarization of neural networks.

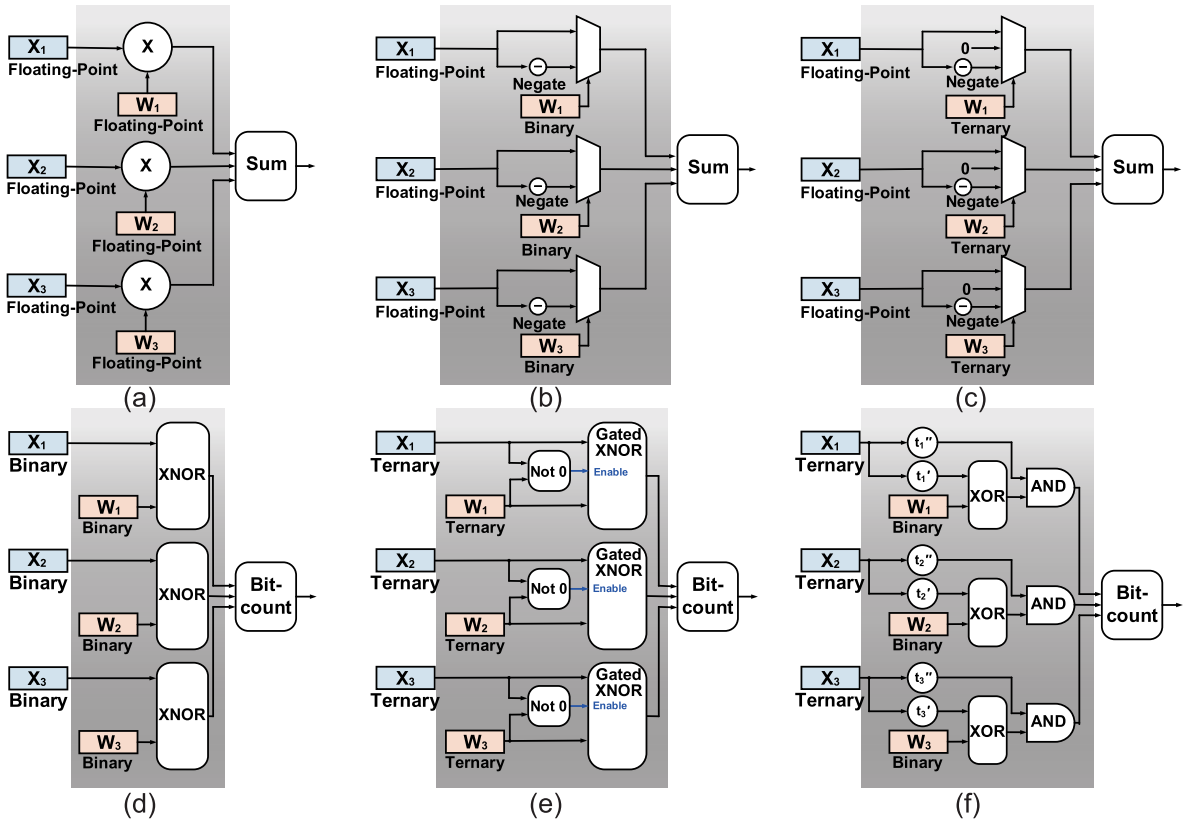


Fig. 2. Overview of the hardware realization of a dot product in (a) conventional DNNs, (b) BWNs, (c) TWNs, (d) BNNs, (e) TNNs, and (f) TBNs.

A. Binarization and Ternarization

Binarization is a form of extreme quantization that compresses values from FP32 to just one bit. While 1-bit values are typically represented as 0 and 1, in the context of neural network weights and activations, which can be either positive or negative, they are usually represented as + 1 and −1, with 0 being mapped to −1. The most common method for this transformation is done by using the Sign function, which preserves the sign of the original floating-point value as follows:

$$\mathrm{Sign}\,(x) = \begin{cases} 1, & \text{if} x \geq 0 \\ -1, & \text{if} x < 0. \end{cases} \tag{2}$$

Ternarization is another form of extreme quantization that reduces values from FP32 to 2 bit (or approximately $\log_2 3 = 1.58$ bit). Unlike the binarization, which maps values to $\{-1, +1\}$ based solely on their sign, ternarization introduces a threshold parameter Δ, which divides values into three levels: $\{-1, 0, +1\}$. This parameter Δ acts as a threshold for determining the output value. The ternarization function, Tern, assigns values as follows: if the input is greater than Δ, the output is set to + 1; if the absolute value of the input is less than or equal to Δ, the output is set to 0; and if the input is less than $-\Delta$, the output is set to −1:

$$\mathrm{Tern}\,(x, \Delta) = \begin{cases} 1, & \text{if} x > \Delta \\ 0, & \text{if} |x| \leq \Delta \\ -1, & \text{if} x < -\Delta. \end{cases} \tag{3}$$

B. Binary Weight Network

A binary weight network (BWN) converts full-precision weights to binary weights while preserving the bit-width of activations, which can be formulated by

$$Y = I \odot W \approx I \odot \left(S_W \cdot \mathrm{Sign}\,(W)\right) = S_W \cdot \left(I \odot \mathrm{Sign}\,(W)\right). \tag{4}$$

In this equation, the Sign function binarizes the full-precision weights, while S_W is the scale factor associated with Sign(W), aimed at reducing quantization error. Since both matrix multiplication ($\odot$) and scale factor computation ($\cdot$) are linear operations, their order can be interchanged without affecting the result, which allows us to rewrite $I \odot (S_W \cdot \mathrm{Sign}(W))$ as $S_W \cdot (I \odot \mathrm{Sign}(W))$. Given that Sign($W$) outputs values in $\{-1, +1\}$, the operation $I \odot \mathrm{Sign}(W)$ simplifies to flipping or retaining the sign of the activation values and accumulating them, as shown in Fig. 2(b). By binarizing weights, this approach not only reduces memory requirement in storing weight parameters, but also lowers computational overhead by replacing expensive multiplications with simple sign flips.

C. Ternary Weight Network

A ternary weight network (TWN) converts full-precision weights to ternary weights while preserving the bit-width of activations, which can be formulated by

$$\begin{aligned} Y = I \odot W &\approx I \odot (S_W * \mathrm{Tern}\,(W, \Delta_W)) \\ &= S_W * (I \odot \mathrm{Tern}\,(W, \Delta_W)). \end{aligned} \tag{5}$$

The TWN differs from the BWN in two aspects. First, unlike the BWN, which uses 1 bit to represent −1 and +1, the TWN introduces an additional value, 0, requiring 2-bit representation. Although this increases the parameter size compared to the BWN counterpart, the inclusion of 0 enables TWNs to achieve higher accuracy. Second, while the BWN employs the Sign function for quantization, the TWN uses a threshold-based ternarization. This method allows the TWN to more accurately capture the distribution of original weights. However, running TWNs in hardware requires additional logic to identify zero weights and gating activations [Fig. 2(c)]. Also, TWNs double the required memory footprint for weights when compared with BWNs since 2 bit is required to store three states.

D. Binary Neural Network

A BNN converts both full-precision weights and activations to binary values, which simplifies the matrix multiplication to

$$\begin{aligned} Y = I \odot W &\approx \left(S_I \cdot \text{Sign}\,(I)\right) \odot \left(S_W \cdot \text{Sign}\,(W)\right) \\ &= S_I \cdot S_W \cdot \left(\text{Sign}\,(I) \odot \text{Sign}\,(W)\right). \end{aligned} \tag{6}$$

Unlike previous methods that focus solely on low-bit quantization of weights, BNNs also quantize activations to 1 bit. This approach replaces multiplication operations with XNOR operations, and the resulting binary outputs are aggregated using popcount operations, as shown in Fig. 2(d), simplifying computations and significantly reducing memory usage by a factor of 32. Despite these advantages, the 1-bit quantization can lead to reduced inference accuracy. To address this, recent BNNs incorporatea advanced techniques, such as modifying underlying architectures [26], [27], allowing a small portion of high-bit operations for error-prone layers [3], [4], minimizing quantization error after binarization [28], [29], and using specialized loss functions [30], [31], to improve accuracy while maintaining their efficiency.

E. Ternary Neural Network

A TNN converts full-precision weights and activations to ternary values, which alters the matrix multiplication to

$$\begin{aligned} Y = I \odot W &\approx \left(S_I \cdot \text{Tern}\,(I, \Delta_I)\right) \odot \left(S_W \cdot \text{Tern}\,(W, \Delta_W)\right) \\ &= S_I \cdot S_W \cdot \left(\text{Tern}\,(I, \Delta_I) \odot \text{Tern}\,(W, \Delta_W)\right). \end{aligned} \tag{7}$$

TNNs utilize a ternary representation with values $\{-1, 0, +1\}$, offering improved accuracy over BNNs. As illustrated in Fig. 2(e), hardware realization of a TNN requires additional gating logic for making the output zero whenever a zero activation or weight is encountered. Despite the increased hardware complexity, the inclusion of "0" allows TNNs to better capture a Gaussian-like distribution of activations/weights, centered near zero, resulting in superior performance relative to BNNs [32], [33], [34].

F. Computational Efficiency of Binary and Ternary Algorithms

The operations in traditional neural networks, as shown in Fig. 2(a), consist of matrix multiplication between the input and weights, with the majority of power consumption during computation arising from full-precision multiplication. For example, FP32 multiplication requires 3.3× more power than FP32 addition, and INT8 multiplication requires about 6.7× more power than INT8 addition [35]. In the case of BWN, as illustrated in Fig. 2(b), the multiplication operation can be replaced by altering the sign of the input based on the sign of the weight using MUX, instead of multiplying the original weights and inputs. TWN, as depicted in Fig. 2(c), introduces an additional value of 0 to the weights, allowing the output to be set to 0 when the weight is 0. Furthermore, if the hardware supports sparsity, zeros in the weights can be excluded from the computation or even memory access, leading to faster inference speed compared to BWN.

In the case of BNN and TNN shown in Fig. 2(d) and (e), the multiplication results of the input and weight are limited to $\{-1, 0, +1\}$, allowing the accumulation operation to be replaced with a popcount instead of the traditional addition used in BWN and TWN. This substitution significantly reduces power consumption. For example, XNOR-Net [5] consumes approximately 58× less power during inference compared to an FP32 network with the same structure. Particularly in the case of analog CIM, there is an advantage in using analog values to process ternary bits at once, unlike digital logic, which requires 2 bit to represent ternary values. In SRAM-based CIM, which can store only 1 bit/cell, a structure like TBN [36] is sometimes used, where ternary analog inputs are processed with binary weights [Fig. 2(f)].

G. Summary

For BWNs and TWNs, weights in the model are quantized to 1 and 2 bit, respectively, minimizing memory usage and replacing power-/area-hungry multipliers with more hardware-efficient multiplexers. Recently, this weight-only quantization approach has been selected as a solution for large language models (LLMs) that are inherently memory-bounded, which requires moving enormous weights from off-chip to on-chip memory, especially during generation tasks [37], [38], [39], [40], [41].

BNNs or TNNs go a step further by quantizing both weights and activations to 1 or 2 bit, not only reducing memory usage but also enhancing computational efficiency by replacing matrix multiplication with XNOR operations. These models are particularly useful in edge computing, where low power consumption and reduced computational resources are critical. In addition, CIM [42], [43], [44], [45] technology can be utilized to run BNNs or TNNs under an extremely constrained power budget, which will be discussed in Section VI.

III. Binarization/Ternarization of CNNs for Vision Tasks

CNNs are not only suited for vision tasks [2], [46], [47], but also widely used as standard networks for processing speech and time-series data [48], [49], [50], leveraging parameter reuse of convolution kernels, making them well-suited for edge device applications. In this article, we first aim to summarize binary/ternary techniques that are optimized for CNNs. To

TABLE I
COMPARATIVE ANALYSIS OF BWNS/TWNS EVALUATED ON IMAGENET-1K DATASET

	Reference	Key Methods	Bit-Width (W/A)	W Value	Model	Acc. (%)
BWN	BinaryConnect [51]	Stochastic binarization on weights	1/32	$\{-1,+1\}$	**A**	35.4
	BWN [5]	Scale factor for reducing binarization errors	1/32	$\{-\alpha,+\alpha\}$	**R**	60.8
	ABC-Net [52]	Linear combination of multiple binary weights	1/32	$\{-\alpha,+\alpha\}$	**R**	62.8
			(1 * 5)/32	$\{-\alpha,+\alpha\}$	**R**	68.3
	BinaryRelax [53]	Continuous regularizer to relax non-convex binarization loss	1/32	$\{-\alpha,+\alpha\}$	**R**	63.2
	LR-net [54]	Training discrete weights using local reparameterization	1/32	$\{-1,+1\}$	**R**	59.9
	ELQ [55]	Regularizing loss perturbation and quantization error	1/32	$\{-\alpha,+\alpha\}$	**R**	64.7
	FleXOR [56]	Weight compression with XOR decoding to achieve $<$ 1-bit	0.8/32	$\{-\alpha,+\alpha\}$	**R**	63.8
			0.6/32	$\{-\alpha,+\alpha\}$	**R**	62.0
	SNN [57]	Clustering binary kernels in a convolutional layer	0.67/32	$\{-\alpha,+\alpha\}$	**R**	63.7
			0.44/32	$\{-\alpha,+\alpha\}$	**R**	60.1
TWN	TWN [58]	Optimal threshold for ternarizing weights	1.58/32	$\{-\alpha,0,+\alpha\}$	**R**	61.8
	TTQ [59]	Separation of positive and negative scale factors	1.58/32	$\{-\alpha,0,+\beta\}$	**R**	66.6
	BinaryRelax [53]	Continuous regularizer to relax non-convex binarization loss	1.58/32	$\{-\alpha,0,+\alpha\}$	**R**	66.5
	[60]	Statistical weight scaling and residual expansion	1.58/32	$\{-\alpha,0,+\alpha\}$	**R**	66.0
			(1.58 * 2)/32	$\{-\alpha,0,+\alpha\}$	**R**	68.1
	LR-net [54]	Training discrete weights using local reparameterization	1.58/32	$\{-1,0,+1\}$	**R**	63.5
	ELQ [55]	Regularizing loss perturbation and quantization error	1.58/32	$\{-\alpha,0,+\alpha\}$	**R**	67.3
	SCA [61]	Adaptive regularization on weights for sparsity control	1.58/32	$\{-1,0,+1\}$	**R**	67.9
	STeP [62]	Pre-defined center-symmetric ternary patterns and Haar-structured features	1.58/32	$\{-1,0,+1\}$	**M**	57.0*

A is AlexNet, **R** is ResNet-18, **M** is MobileNet-V2.
$*$ is tested on TinyImageNet.

avoid redundancy with existing BNN literature [13], [14], we review BNN articles published after 2023 and focus more on BWN, TWN, and TNN, which have received relatively less attention.

A. BWN/TWN for CNNs

Prior work on BWNs or TWNs employs methods to minimize the error that occurs from quantizing weights into extremely low-bit representations. These methods include reducing the quantization error between the original floating-point weights and the binary/ternary weights [5], [51], [58], [59], using weight regularization techniques optimized for low-bit quantization [55], redesigning the network architecture [52], or incorporating a new loss function to improve the learning process [61]. Table I summarizes the key methodologies of BWN/TWN.

1) Weight Binarization in CNNs: BinaryConnect [51] is the first BWN work that improves the performance of deep learning models by binarizing the floating-point weights to $\{-1,+1\}$ during the forward pass, while using the original floating-point weights during the backward pass for training. The BWN proposed in XNOR-Net [5] significantly improves the performance of networks compared to BinaryConnect by representing the weights as $\{-\alpha,+\alpha\}$, where α tries to minimize the binarization error. BinaryRelax [53] addresses the issue of the nonconvex loss landscape caused by the traditional binarization function by using a continuous regularizer to relax the landscape. By smoothing out the loss surface, this approach finds optimal binarized weights more effectively. ABC-Net [52] not only binarizes the existing network structure, but also appends multiple binarization blocks (called *bases*) that are combined through linear combination. While this approach increases the computation overhead, it achieves a similar accuracy to that of the original full-precision network.

FleXOR [56] and subbit neural network (SNN [57]) reduce the size of each weight, on average, to less than 1 bit by compressing weights. FleXOR stores binary weights in a compressed/encrypted format (<1 bit on average) while they are decoded on-the-fly using XOR gates to perform required binary operations. For instance, instead of directly representing ten weights with ten individual 1-bit values, FleXOR uses six bits to represent them and decodes them by using a set of XOR gates. This setup allows the ten weights to be out of 2^6 possible combinations, effectively representing ten weights with just 0.6 bits/weight (i.e., 6 bit for storing ten weights). The SNN [57] designs multiple subsets for a 3×3 binarized convolution filter and trains the network with the awareness of these subsets, allowing the weights to be stored with a bit-width smaller than 1 bit. For instance, while a standard 3×3 binarized convolution filter has $2^9 = 512$ possible patterns, SNN reduces this by only using frequently appearing subsets with $2^6 = 64$ combinations. Then, SNN fine-tunes the network by only using 64 binary kernels, resulting in a 0.67-bit network without hurting the accuracy much.

2) Weight Ternarization in CNNs: A TWN not only introduces a value of 0 in addition to the binary values $\{-1,+1\}$ with scale factor(s), but also relies on the threshold value (Δ) that determines which ternary value to assign. Li et al. [58] have introduced the first TWN, where the scale factor α is optimized while setting the threshold value $\Delta = 0.75 \cdot |\mathbf{W}|$ for original weights $\mathbf{W}$ assuming normal distribution. This approach has demonstrated superior performance compared to BinaryConnect [51] and the BWN model from XNOR-Net [5]. Trained ternary quantization (TTQ [59]) extends the traditional TWN by using separate positive and negative scale factors, i.e., α and β, respectively, which results in higher accuracy than the TWN with a single scale factor. However, employing distinct scale factors for positive and negative weights introduces computational overhead, requiring more complex computing

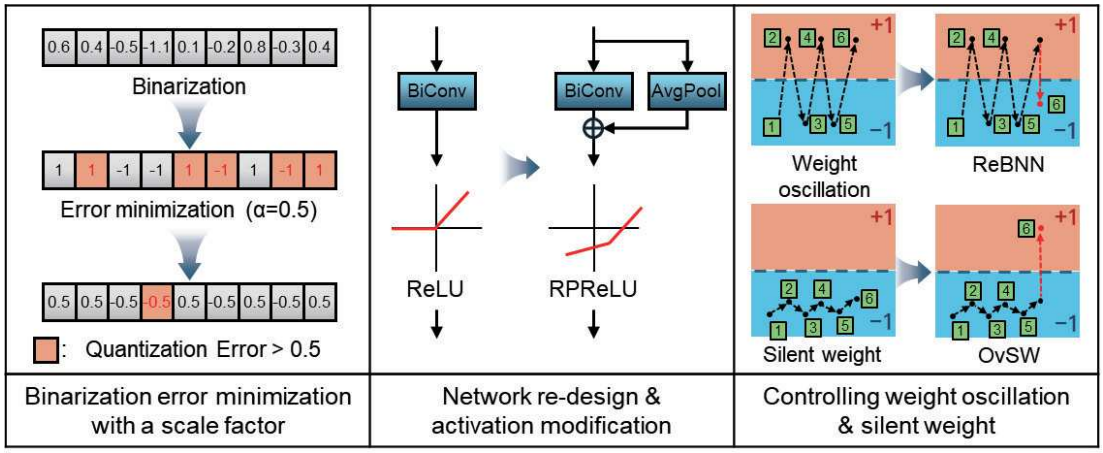


Fig. 3. Development direction of BNNs from introducing a scale factor to controlling the training quality.

TABLE II
COMPARATIVE ANALYSIS OF BNNS/TNNS EVALUATED ON IMAGENET-1K DATASET

Reference	Key Methods	W Value	A Value	Model	Acc (%)
BNN [63]	Simple binarization of weights and activations	$\{-1,1\}$	$\{-1,1\}$	**A**	27.9
XNOR-Net [5]	Analytical scaling factors for reducing binarization error	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**A**	44.2
				R	51.2
XNOR-Net++ [64]	Decomposition of a scaling factor into multiple dimensions	$\{-1,1\}$	$\{-\Gamma,\Gamma\}^\star$	**R**	57.1
Signed-Bin. [65]	Separation of positive and negative weights with zeros	$\{-\alpha_1,0\}$, $\{0,\alpha_2\}$	$\{-\beta,\beta\}$	**R**	66.2
AdaBin [66]	Layer-wise adaptive binary sets for weights and activations	$\{-\alpha_1,\alpha_2\}$	$\{-\beta_1,\beta_2\}$	**R**	66.4
Bi-Real-Net [29]	Approximation of the derivative of the sign function	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**BR**	56.4
ReActNet [30]	Generalized activation functions with learnable coefficients	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**RA**	69.4
INSTA-BNN [67]	Sign function with an input-aware dynamic threshold	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**RA**	72.2
RAD-BNN [68]	Information entropy enhancement to regulate activation distribution	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**R**	65.6
PokeBNN [4]	A binary convolution block with a residual path and SE-based scaling	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**C**	73.4
ReBNN [69]	Weighted reconstruction loss to control weight oscillation	$\{-\alpha,\alpha\}$	$\{-\beta,\beta\}$	**R**	66.9
DBNN [70]	Dilution of weights with small magnitude to solve a sign flip problem	$\{-\alpha,0,\alpha\}$	$\{-\beta,\beta\}$	**R**	59.3
OvSW [71]	Adaptive gradient scaling to handle silent weights	$\{-1,1\}$	$\{-\Gamma,\Gamma\}^\star$	**R**	66.6
TNN [72]	Ternarization of both weights and activations	$\{-1,0,1\}$	$\{-1,0,1\}$	**V**	87.9†
GXNOR-Net [73]	Hardware acceleration using gated XNOR units	$\{-1,0,1\}$	$\{-1,0,1\}$	**V**	92.5†
TBN [36]	Use of ternary activations and binary weights	$\{-\alpha,\alpha\}$	$\{-1,0,1\}$	**R**	55.6
RTN [34]	Reparameterization of tensors to reduce activation squashing	$\{-\alpha,0,\alpha\}$	$\{-\gamma+\beta,$ $\beta,$ $\gamma+\beta\}$	**R**	64.5
FATNN [32]	ReLU-aware ternary quantization; activation after ReLU is $(0,\beta,2\beta)$	$\{-\alpha,0,\alpha\}$	$\{-\beta_1,0,\beta_1\}$, $\{0,\beta_2,2\beta_2\}$	**R**	66.0
TRQ [74]	Stem-residual quantization for minimizing ternarization error	$\{-\alpha,0,\alpha\}$	$\{-\beta,0,\beta\}$	**R**	65.7
STTN [75]	Soft threshold for ternarization to arbitrary bit-width to mitigate error	$\{-\alpha,0,\alpha\}$	$\{-\beta,0,\beta\}$	**R**	67.1

* Γ is the fused and trainable scale factor for both **A** and **W**.
† Tested on CIFAR-10 dataset.
A is AlexNet, **R** is ResNet-18, **BR** is Bi-Real-Net18, **RA** is ReActNet-A, **V** is VGG-7, **C** is task-specific custom model.

logic [61]. He et al. [60] have proposed iterative statistical weight ternarization that iteratively trains the layerwise scaling factor for suppressing ternarization error. They further improve the accuracy by presenting residual expanded layers, which add extra layer(s) to the existing convolution layer with the overhead of computational cost and memory footprint.

Other than carefully setting the scaling factor (α or β) and the threshold value (Δ) for ternarization, one may come up with a better training algorithm or a novel loss/regularization term for TWNs. LR-Net [54] achieves higher accuracy compared to traditional training methods by using the local reparameterization trick, i.e., extracting statistics by using preactivations instead of weights, to train on discrete binary or ternary weights. Additionally, LR-Net demonstrated that setting the output to zero for results with low-mean and low-variance weights in a ternary network is crucial for performance improvement. ELQ [55] improves the performance of BWN and TWNs by adding an explicit loss term during training to account for the errors between full-precision weights and low-bit weights. SCA [61] improves the performance of TWNs by adding a regularization term that directly controls the proportion of zeros, which was not considered in traditional TWNs. Also, by ternarizing weights without a scale factor, i.e., $\{-1,0,+1\}$, the SCA makes hardware for TWNs easier to

implement. STeP [62] defines 3×3 convolution kernels to have a center-symmetric local binary pattern (CS-LBP) and Haar-structured feature (HSF) with $\{-1, 0, 1\}$. By making predefined ternary weights unchanged, 1×1 real-valued convolution kernels are trained. This approach effectively reduces the number of trainable parameters by 56.1% on average with a minor accuracy drop of 3.1%.

B. Binary/Ternary Neural Network for CNNs

BNNs or TNNs not only reduce the bit-width of weights, but also reduce that of activations to an extremely low bit-width, making the entire computation into gate-level operations, which significantly increases the computational efficiency. However, it has been observed that quantizing activations to extremely low-bit precision results in significant accuracy degradation, unlike BWNs or TWNs. As shown in Fig. 3, prior work on BNNs and TNNs have presented various techniques to address this issue, such as reducing quantization error [5], [32], [36], [65], [66], [72], [73], [74], [75], redesigning activation functions or models [4], [30], [34], [67], [68], [76], and introducing novel gradient update schemes [69], [70], [71]. Table II further summarizes these approaches.

1) Binarized CNNs: Hubara et al. [63] have proposed the first BNN architecture, explicitly demonstrating the efficiency gain by binarizing both weights and activations. XNOR-Net [5] took a step further by using $L1$-norm to calculate scale factors for weights and activations that minimize binarization error, resulting in 16.3% accuracy improvement over the original BNN. XNOR-Net++ [64] improves upon the XNOR-Net by fusing scale factors for activations and weights into a single trainable factor Γ, rather than analytically calculating it. In addition, it decomposes the scale factor and learns a rank-1 factor for each tensor dimension. This approach achieves 5.9% higher accuracy compared to the original XNOR-Net [5]. Signed-Binarization [65] improves the accuracy of BNN by keeping two different kernel categories: one consisting of 0 and negative values ($-\alpha_1$), and the other consisting of 0 and positive values ($+\alpha_2$). This method introduces zero into weights without increasing the bit-width, i.e., binary weights but effectively ternary, significantly improving the accuracy. AdaBin [66] improves performance by moving away from using a symmetric scale factor, instead determining the optimal scale factors for -1 and $+1$, respectively, for each layer. Although this approach slightly increases the computational cost compared to traditional BNNs, it achieves significantly higher accuracy.

Other than playing with scale factors to minimize the binarization errors in activations and weights, one may design a new BNN architecture, e.g., adding real-valued skip connections, modifying the activation function, or smoothing the binarization function. Bi-Real-Net [29] improves the accuracy by using an *ApproxSign* function, which approximates the traditional sign function with a differentiable polynomial function to enable better gradient approximation. In addition, by allowing skip connections from real-valued activations after the BatchNorm layer, Bi-Real-Net improves the representational capability. ReActNet [30] embeds learnable coefficients in activation functions to control data distributions within the BNN. It introduces *ReAct-Sign* and *ReAct-PReLU*, which replace the traditional *Sign* and *PReLU* functions, respectively, by converting static thresholds of the *Sign* function into learnable parameters. This allows the activation distribution to be shifted and reshaped for binary operations. INSTA-BNN [67] achieves higher accuracy than ReActNet by dynamically adjusting the threshold of the *Sign* function based on statistics of a given input instance, i.e., instancewise mean. This instance-aware threshold control allows a more precise representation of activations even with binary values. Similarly, RAD-BNN [68] improves the accuracy by dynamically scaling the input feature map before the *PReLU* function. This dynamic scaling acts as a form of regularization, leading to improved results. PokeBNN [4] demonstrates the best practice of BNNs so far by presenting a binarization-friendly convolution block, which incorporates a real-valued skip connection and a squeeze-and-excitation (SE) block with learnable per-channel scaling.

Orthogonal to introducing proper scale factors and/or modifying activation functions, some researchers have focused on inherent challenges in training BNNs, e.g., frequent weight oscillation [69], [70], or the presence of silent weights [71]. When training BNNs, small weights change their signs frequently, i.e., weight oscillation, due to the magnified impact of the scale factor α on weight gradients. To address this, ReBNN [69] introduces the concept of a balanced parameter γ, which constitutes the prime factor in weighted reconstruction loss that controls the weight oscillation throughout the training. Diluted BNN (DBNN [70]) also solves the problem of small weights, i.e., near zero, becoming either -1 or $+1$, resulting in a large binarization error. In DBNN, small weights in the original model are set to zero (i.e., diluted or sparsified), which naturally suppresses frequent sign flips of small values during BNN training. On the other hand, OvSW [71] tackles the issue of silent weights, which are weights that are never updated during training. The OvSW presents adaptive gradient scaling (AGS), which amplifies gradients of a weight group for each output channel when its Forbenius norm is significantly smaller than that of the corresponding weights to excite those silent weights.

2) Ternarized CNNs: The limited expressiveness of BNNs, i.e., 2^N different states for an N-dimensional binary vector, has motivated the research on TNNs. TNN [72] is the first work that ternarizes both weights and activations allowing $\{-1, 0, +1\}$. The original TNN work tries to find low and high thresholds for ternarization by analyzing the output distribution of the full-precision counterpart. The increased level of expressiveness from 2^N to 3^N allows TNNs to achieve higher accuracy compared to early BNNs. Gated XNOR-Net (GXNOR-Net [73]) modifies the XNOR-Net structure to account for the ternary data, which add 0 to both weights and activations. This not only allows for a more diverse representation of data distribution, but also enables a more efficient hardware operation by skipping computations by gating zero values. The gating logic can be realized by simply adding a multiplexer on top of XNOR-based BNN hardware, enabling fast and efficient inference [Fig. 2(e)]. As a variant to a TNN, TBN [36] ternarizes activations

while binarizing weights to strike a balance between accuracy and computational efficiency, similar to GXNORNet, which utilizes gate logic by binarizing only the weights while keeping the activations ternary. TBN replaces the dot product between two vectors, i.e., activation and weight, with XOR and AND logic operations for multiplications, followed by the bitcount for accumulations.

Both RTN [34] and FATNN [32] introduce changes to activations of conventional TNNs. RTN dynamically changes the squashing range of ternarization by scaling and shifting the ternarized activations after the BatchNorm layer. On the other hand, FATNN modifies the activation values right after ReLU, recognizing that ReLU outputs nonnegative values. Instead of using $\{-\beta, 0, \beta\}$, FATNN used $\{0, \beta, 2\beta\}$ for activations after ReLU, improving its performance. TRQ [74] uses a stem-residual quantization, which handles weight quantization through residuals. This method reduces quantization error more effectively than conventional ternary functions, leading to accuracy improvement. Rather than relying on a hard threshold for ternarization, STTN [75] presents the soft threshold, which is automatically determined by the model. Also, each ternary value is computed by the combination of two binary values, which deviates from other TNN works.

3) Comparison of Various BNNs/TNNs: Fig. 4 shows the comparison of various BNNs and TNNs in terms of operations (OPs) (= 1/64 binary operations (BOPs) + floating-point operations (FLOPs)) and accuracy. This shows that by creating a model architecture optimized for BNN/TNN, higher accuracy can be achieved even with fewer OPs. By moving from the standard ResNet-18 [2] to ReActNet-A [30], i.e., a BNN-optimized architecture, and further to the PokeBNN [4], which converts some floating-point operations to quantized integer operations, higher accuracy is achieved with fewer OPs even when using the same quantization algorithm. Therefore, we must not only quantize models to low-bit precision, but also change model architectures that are optimized for the specific hardware and task at hand.

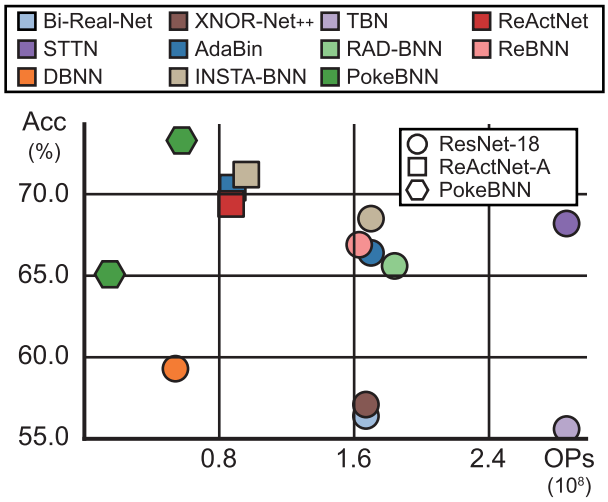


Fig. 4. Comparison of various BNN/TNN models (dataset: ImageNet-1K).

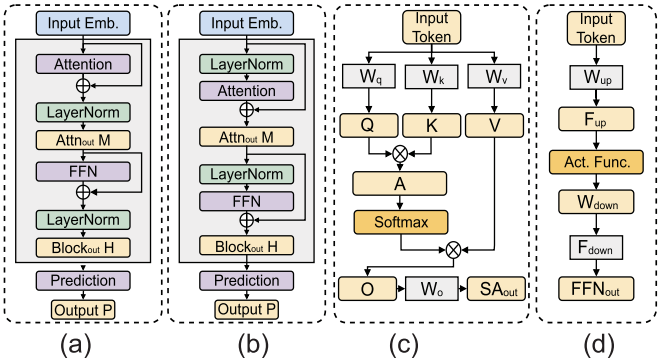


Fig. 5. Overview of Transformer architectures: BERT [15] and ViT [18] for language and vision tasks, respectively. (a) BERT. (b) ViT. (c) Attention. (d) FFN.

IV. Binarization/Ternarization of Transformers for Language/Vision Tasks

The Transformer is a new architecture in deep learning that uses the attention mechanism [77] not only to extract intrinsic features in input data, but also to learn the relationship between different data points. It is now becoming a mainstream model for a variety of AI applications, including speech [21], vision [18], [19], [20], [78], language [15], [16], [17], and multimodality [79], [80].

As shown in Fig. 5, the Transformer architecture is broadly divided into two main components: the attention block (Attention) and the feed-forward network (FFN) block. The attention block is responsible for computing the relationship between input tokens, which looks at every token pair to extract the relevance

$$Q = I \odot W^Q, K = I \odot W^K, V = I \odot W^V \tag{8}$$

$$\text{Attention}\,(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathrm{T}}}{\sqrt{D_K}}\right) \odot V = A \odot V \tag{9}$$

where $I \in \mathbb{R}^{N \times H}$ is the embedded input tokens, $W^Q, W^K, W^V \in \mathbb{R}^{H \times D_k}$ are the weights in the attention block for generating Q, K, and V matrices, respectively, N is the sequence length, H is the input dimension, and D_k is the hidden dimension. $S = QK^{\mathrm{T}}$ is often called the attention score, and softmax$(QK^{\mathrm{T}}/(D_K)^{1/2})$ is often called the attention matrix, where softmax outputs the probability of each row in the attention score S. At this stage, the attention scores are scaled by $(D_k)^{1/2}$, where D_k is the dimension of the key vectors. This scaling helps smoothing the attention score preventing the gradient vanishing. Then, the resulting attention matrix is being multiplied with V matrix. This entire process is called scaled dot-product attention. In typical Transformer models, multiple scaled dot-product attention blocks are used, called multihead attention (MHA).

Then, the output from the MHA block is fed into an FFN. The FFN layer applies a nonlinear function to increase the expressiveness of the intermediate embedding, where its operation is given by

$$\text{FFN}\,(X) = \sigma\left(X \odot W_{\text{up}} + b_{\text{up}}\right) \odot W_{\text{down}} + b_{\text{down}}. \tag{10}$$

In the FFN, each token is independently processed using two fully connected layers, with parameters W_{up} and W_{down}, and a nonlinear function. To comprehend more complex data

TABLE III
COMPARATIVE ANALYSIS ON BINARIZATION/TERNARIZATION OF BERT

Reference	Bit-Width (E/W/A)	Size (MB)	GFLOPs	STS-B	MRPC	RTE	QQP	QNLI	SST-2	COLA	MNLI (m/mm)	Avg.
BERT_{base} [15]	32-32-32	418.0	22.5	88.8	86.5	66.8	91.4	91.3	92.9	55.8	83.4/83.6	82.3
TernaryBERT [84]	1.58-1.58-8	28.0	6.4	—	87.5	68.2	90.1	—	—	50.7	83.3/83.3	—
BinaryBERT [85]	1-1-8	16.5	3.1	88.6	85.5	72.2	91.2	91.5	92.6	53.4	84.2/84.7	82.7
$ETSPQ^{\dagger}$ [86]	1.58-1.58-8	28.0	6.4	89.8	88.0	74.7	91.4	92	93.9	58.8	84.8/—	84.5
	1-1-8	13.4	3.1	89.6	88.2	74.7	91.3	91.6	93.9	55.6	84.2/—	83.6
BiBERT [87]	1-1-1	13.4	0.4	33.6	72.5	57.4	84.8	72.6	88.7	25.4	66.1/67.5	63.2
BiT [88]	1-1-4	13.4	1.5	86.3	86.8	91.3	87.8	91.3	91.5	42.0	83.6/84.4	79.5
	1-1-2	13.4	0.8	82.2	78.4	58.1	87.1	89.3	90.8	32.1	82.1/82.5	75.0
	1-1-1	13.4	0.4	72.0	79.9	62.1	85.4	86.4	89.9	32.9	79.5/79.4	73.5
BiPFT [89]	1-1-2	14.9	0.8	87.0	84.1	66.1	89.0	86.6	88.1	36.2	77.0/76.9	76.8
	1-1-1	14.9	0.4	80.2	76.2	66.1	83.7	81.7	86.2	22.9	69.5/70.6	70.8

† Using data augmentation.

structures and contexts, we may stack multiple Transformer blocks while adding a residual connection and normalization within a block.

Both BERT [15] and ViT [18] enhance their performance by utilizing self-attention mechanisms. BERT leverages self-attention to capture contextual relationships between tokens/words in a sentence, while ViT applies self-attention to image patches to model the relationship between other image patches at different regions. However, due to a large number of parameters and high computational complexity of the Transformer, it is extremely challenging to deploy it on mobile devices, making model compression essential. Quantization not only efficiently reduces the model size, but also has the advantage of converting computations originally performed in FP32 to INT. Unlike conventional convolution blocks, Transformers contain operations that are difficult to quantize, such as softmax and Gaussian error linear unit (GELU), requiring deeper consideration when selecting and applying quantization algorithms [81], [82]. Thus, this section has two objectives: 1) to review the application of binarization and ternarization to support diverse Transformer architectures and 2) to introduce various techniques to handle these challenging operations at extremely low-bit precision.

A. Binarization/Ternarization of BERT for Language Tasks

The bidirectional encoder representations from Transformers (BERT [15]) are based on the encoder structure of the Transformer. Unlike RNNs, which capture relationships in a unidirectional manner, BERT uses a bidirectional model structure, allowing it to achieve a better understanding of input sentences. One of its key advantages is that after being pretrained on a large amount of data, it can be fine-tuned for specific tasks, called downstream tasks, delivering high performance even with a small amount of task-specific data. However, models that undergo pretraining on large datasets and are fine-tuned on task-specific small datasets require a large model size due to the need for a comprehensive understanding of diverse data. For example, the BERT-based model demands over 400 MB of storage space and more than 22 GFLOPs, making it burdensome to deploy the model directly on mobile devices. Therefore, the goal is to reduce the model size and the number of computations through low-bit quantization. Similar to quantization of CNNs, as shown in Table III, we can apply binarization/ternarization to only weights or both weights and activations.

1) Weight Binarization/Ternarization in Encoder-Based Transformer: TernaryBERT is the first article to attempt ternarizing weights, which take values in $\{-1, 0, +1\}$, of a pretrained BERT. For the activations, TernaryBERT quantizes them to INT8 using the min–max quantization, which handles asymmetric distribution better. To reduce the bit-width of the weights, this article compares two widely accepted methods for weight ternarization: the TWN [58] approach, which analytically defines a scaling factor and a threshold, and the LAT [83] method, which directly learns the scaling factor via backpropagation by leveraging diagonal elements in the Hessian of the loss. TernaryBERT also evaluates the granularity of ternarization by testing layerwise, i.e., per each weight, and rowwise scaling factors, i.e., per row of each weight.

To minimize the accuracy loss due to ternarization, i.e., extreme quantization, TernaryBERT utilizes knowledge distillation by making a full-precision model as a teacher and the ternarized model as a student. As the first loss term, mean squared error (mse) is used to minimize the distance between outputs of embedding layers and attention blocks of the teacher (E_t, H_t) and the student (E_s, H_s), where E is the `Input Emb` and H is the $\texttt{Block}_{\texttt{out}}$ in Fig. 5(a). It also introduces another loss term to match attention scores of the teacher (A_t) and those of the student (A_s), where A is the output of $Q \cdot K^{\mathrm{T}}$ in Fig. 5(a). As the final loss term, it distills the knowledge from the prediction layer, i.e., the final logits, by using soft cross-entropy (*SoftCE*). The total loss function can be formulated as follows:

$$\begin{aligned}\text{Loss}_{\text{TernBERT}} = {} & \text{mse}\,(E_t, E_s) + \text{mse}\,(H_t, H_s) \\ & + \text{mse}\,(A_t, A_s) + \text{SoftCE}\,(P_t, P_s)\,. \end{aligned} \tag{11}$$

BinaryBERT [85] suggests a ternary weight splitting technique to address the significant performance drop observed when directly binarizing the BERT model. In this approach, the model is first trained using ternary weights, which cause less performance degradation than binary weights during training. Afterward, the learned ternary weights are split into two

binary weights without any performance degradation. Furthermore, unlike the distillation presented in TernaryBERT [84], BinaryBERT distills the outputs of the attention block, i.e., M in Fig. 5(a), instead of attention score A, which improves the overall performance. Therefore, the total loss for training BinaryBERT becomes

$$\text{Loss}_{\text{BinBERT}} = \text{mse}(E_t, E_s) + \text{mse}(H_t, H_s) + \text{mse}(M_t, M_s) + \text{SoftCE}(P_t, P_s). \quad (12)$$

ETSPQ [86] progressively quantizes weights, i.e., 32 bit→ 16 bit→ 8 bit→ 4 bit→ 2 bit→ 1 bit, ensuring better retention of performance during the binarization/ternarization process. ETSPQ also progressively quantizes activations to either 8 or 4 bit after settling down the bit-width of weights. In addition, it empirically demonstrates that data augmentation is essential for enhancing the performance of low-bit quantization for BERT. By incorporating various data augmentation techniques, the model becomes more robust and capable of maintaining accuracy even when using fewer bits for the weight representation. This highlights the importance of training techniques beyond just quantization methods for improving low-bit models.

2) Weight-Activation Binarization/Ternarization in Encoder-Based Transformer: BiBERT [87] is an algorithm designed for 1-bit quantization on both weights and activations. To achieve this, BiBERT introduces a new structure as an attention block optimized for binarization, named biattention. In the biattention structure, query Q, key K, and value V matrices are quantized to 1 bit, prior to computing the attention score and the final output of the attention block. The computed attention score is, then, binarized, where values exceeding a certain threshold are set to 1, and values below the threshold are set to 0. The threshold is carefully chosen to maximize the entropy of the attention score, thereby improving the performance of the biattention structure. Also, it proposes a novel distillation method, called direction-matching distillation (DMD), which is optimized for low-bit activation quantization. It ensures that the quantized queries (Q_s), keys (K_s), and values (V_s) in the student model maintain the same directional alignment as the full-precision queries (Q_t), keys (K_t), and values (V_t) from the teacher model. This is achieved by reducing the difference between the similarity pattern matrices of the student and the teacher, ensuring that the binarized model follows the directional patterns of the full-precision model. The similarity pattern matrices and DMD loss are defined as

$$P_Q = \frac{Q \odot Q^{\text{T}}}{||Q \odot Q^{\text{T}}||}, P_K = \frac{K \odot K^{\text{T}}}{||K \odot K^{\text{T}}||}, P_V = \frac{V \odot V^{\text{T}}}{||V \odot V^{\text{T}}||} \quad (13)$$

$$L_{\text{DMD}} = \sum_{f \in \Gamma} ||P_{f_t} - P_{f_s}|| \quad (14)$$

where $\|\cdot\|$ is the $L2$ distance and $\Gamma = \{Q, K, V\}$. This approach helps the quantized student model learn effectively from the teacher, preserving important relational information in the attention mechanism. The final loss function in BiBERT is expressed as

$$\text{Loss}_{\text{BiBERT}} = \text{mse}(H_t, H_s) + \text{SoftCE}(P_t, P_s) + L_{\text{DMD}}. \quad (15)$$

Similar to BiBERT, BiT [88] binarizes the attention score for effective binary quantization. Other than that, the activation function used in BERT, i.e., GELU, is replaced with ReLU, as GELU is deemed unsuitable for BNNs. Since ReLU outputs nonnegative values, BiT binarizes the outputs differently based on their value ranges: the outputs of ReLU and softmax, which are in $\{0, 1\}$, are binarized accordingly, while outputs that include negative values are binarized into $\{-1, 1\}$. Furthermore, instead of directly quantizing from full-precision weights and activations (W32A32) to binary weights and activations (W1A1), BiT applies a multistep distillation approach: transitioning from W32A32 to W1A2 and then to W1A1. This intermediate step allows for a smoother transition, similar to ETSPQ [86], and helps minimize performance degradation during binarization.

BiPFT [89] addresses the issue of performance degradation during low-bit quantization of pretrained full-precision networks for downstream tasks, which can be sensitive to hyperparameters. To mitigate this problem, BiPFT proposes creating a binarized pretrained foundation model. To minimize the quantization error that arises during this process, BiPFT employs a method that incorporates low-rank estimation. This approach allows for a small amount of full-precision operations to be added strategically within the network. By doing so, it helps reduce the impact of quantization error, preserving the model's performance while benefiting from the efficiency of low-bit quantization. This technique ensures that even with binarization, the pretrained model retains a high level of accuracy for downstream tasks.

In summary, as shown in Fig. 6, earlier approaches aimed to bridge the representation gap between the full precision and quantized networks using several strategies. These included knowledge distillation [84], [85], [87] to transfer knowledge, DMD [87], [88] to improve model expressiveness by aligning attention maps, and a multistep distillation to minimize performance loss through gradual quantization [86], [88].

B. Binarization/Ternarization of ViT

Due to the significant performance gain of a Transformer on language tasks, it has been modified to target vision-related tasks as well.

TerViT [90] is the first weight ternarization work for ViT. TerViT improves performance by observing that each channel in the Transformer block exhibits different distributions, thus applying per-channel scale factors and thresholds. In addition, instead of directly quantizing from FP32 to ternary, the model first undergoes 8-bit quantization training, followed by a second quantization step into ternary weights, which further enhances performance.

Bi-ViT is a modified form of the binarization method used in Bi-Real-Net [29], adapted for ViTs. When using the original Bi-Real-Net method, gradient mismatch occurs during the self-attention process. To minimize this mismatch, Bi-ViT uses a learnable headwise scale factor (LSF), which directly learns the scale factor of each multihead activation and modifies gradient calculation. Additionally, to minimize the information loss that occurs when binarizing the attention scores, it uses

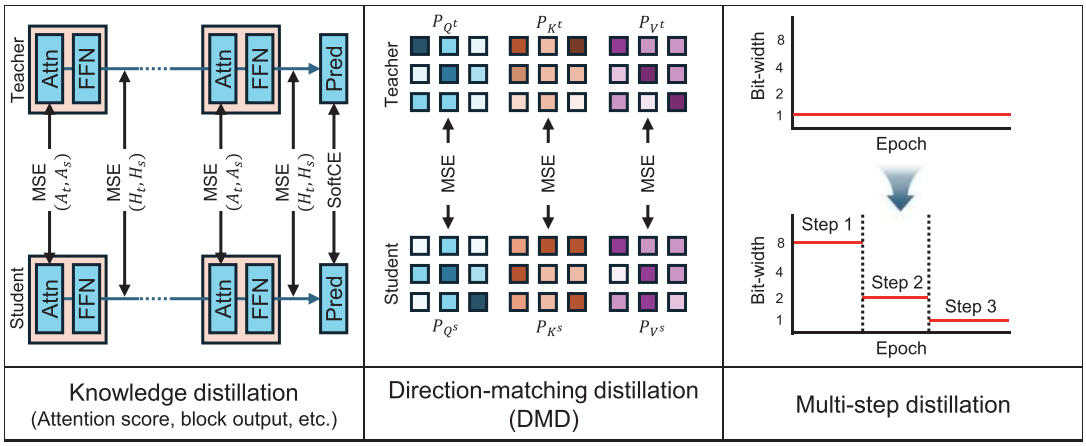


Fig. 6. Development direction of binarized (or ternarized) Transformers from leveraging knowledge distillation to multistep distillation.

ranking-aware distillation (RD), where attention scores are ranked and then distilled.

BiViT [92] is an algorithm designed to solve the problem that arises when binarizing the softmax output, where it fails to accurately follow the original bell-shaped distribution. In the original approach, the sign function was applied to $Q \odot K^{\mathrm{T}}$, converting positive values to 1 and negative values to 0 for softmax binarization. However, this method did not properly represent the bell-shaped distribution of the softmax. To address this, BiViT modifies the process by changing only the values that exceed a certain threshold in the softmax output to 1, making it better follow the original softmax distribution. Furthermore, significant performance degradation was observed when all layers were fully binarized, thus presenting a two-step training process. First, only the self-attention layer is binarized to minimize information loss in the FFN layer, and later, the entire network is binarized and trained to improve the model's final performance.

Si-BiViT [93] identifies the issue of decreasing information interaction between tokens during the binarization process and addresses this problem by appending an additional MLP layer at each FFN block, called the spatial interaction (SI) module. These SI modules can be easily attached to any ViT model. Furthermore, to ensure that the original network's information is not lost during training, two additional training methods were employed. First, the entire network, except for the SI module, is binarized, and the model is trained by minimizing the cross-entropy loss between the predicted and true labels, as well as by distilling features from the original FP32 model to ensure that the learned features closely follow the original network. Then, the SI module is added, and the final network is being trained using cross-entropy loss between the predicted and true labels.

BinaryViT [94] proposes methods to address the issue of weight oscillation, where weight values near the binarization threshold frequently change. To reduce the weight oscillation, a gradient regularization scheme (GRS) is applied to weights near the binarization threshold. Also, accuracy is improved by using an activation shift module (ASM), which dynamically adjusts the activation using two fully connected (FC) layers before applying the sign function, minimizing the information distortion caused by significant changes across channels.

BVT-IMA [95] aims to address issues in the self-attention mechanism of BinaryViT. It points out the excessive information loss that occurs during the binarization of softmax output in existing methods and proposes a novel approach to improve this process. The key idea is to replace the softmax operation with a learnable lookup table (LUT). In this approach, the result of $Q \odot K^{\mathrm{T}}$ is mapped to the values in the LUT. This minimizes information loss in the attention block and, as a result, improves the model's performance. This method enhances the expressiveness of binary ViTs without adding extra computational cost and is particularly effective in mitigating attention shifting effects.

BHViTAQ11 [96] mitigates the performance degradation observed in binary ViTs by introducing four key techniques. First, it incorporates skip connections into each binarized operation to minimize information loss. Second, BHViT adopts a hybrid architecture [97], unlike previous BNN-based ViTs that retain the original structure, using convolutional layers in the early stages to reduce computational cost and attention layers in the later stages to improve accuracy. To further enhance the representational power of attention, BHViT introduces additional window-based tokens alongside the standard patch-based tokens, enabling the network to process a richer set of inputs. Third, the FFN layers are augmented with shift and pooling operations to facilitate more effective information propagation. Finally, to suppress weight oscillations that may arise with the Adam optimizer, BHViT applies $L1$-regularization to the weights, thereby stabilizing updates and minimizing abrupt changes. Table IV provides a performance comparison for above methods of applying binarization/ternarization.

V. BNN/TNN Acceleration in Edge Devices

This section is organized into Section V-A–V-C to introduce hardware acceleration strategies for BNNs and TNNs. First, we introduce bit-packing, a technique for accelerating low-bit quantization on commercial hardware, which improves

TABLE IV
COMPARATIVE ANALYSIS ON BINARIZATION/TERNARIZATION OF VIT

Reference	Model	W/A bit-width	Size (MB)	GOPs	Acc (%)
FP32	DeiT-Tiny [78]	32 / 32	22.7	1.1	72.2
	Swin-Tiny [20]	32 / 32	113.2	4.51	81.2
TWN [58]	DeiT-Tiny [78]	1.58 / 8	1.6	-	64.4
	Swin-Tiny [20]	1.58 / 8	7.7	-	73.7
BiBERT [87]	Swin-Tiny [20]	1 / 1	4.2	0.2	41.9
TerViT [90]	DeiT-Tiny [78]	1.58 / 8	1.6	-	66.6
	Swin-Tiny [20]	1.58 / 8	7.7	-	77.5
Bi-ViT [91]	Swin-Tiny [20]	1 / 1	4.2	0.8	55.5
BiViT [92]	Swin-Tiny [20]	1 / 1	6.3	0.8	58.7
	Swin-Tiny [20]	1 / 1 (MHA) + 1 / 32 (FFN)	6.3	1.6	70.8
Si-BiViT [93]	Swin-Tiny [20]	1 / 1	9.9	0.8	63.8
BinaryViT [94]	Swin-Tiny [20]	1 / 1	5.9	0.3	63.8
BVT-IMA [95]	Swin-Tiny [20]	1 / 1 (MHA) + 1 / 32 (FFN)	6.3	-	72.0
BHViT [96]	Custom	1 / 1	3.5	0.8	68.4
	Custom	1 / 1 + 32 / 32 (Downsample)	3.5	1.5	70.4

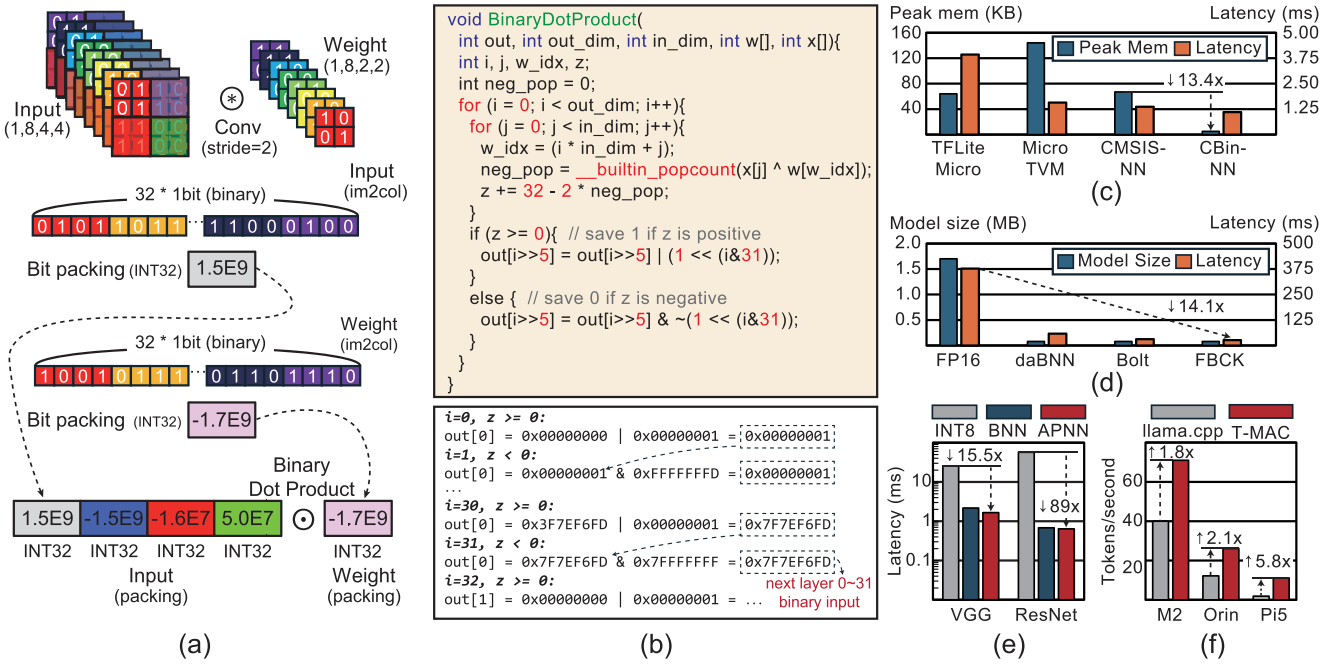


Fig. 7. (a) Overview of binary convolution with bit packing. (b) Code snippet implementing the binary dot product. (c) Comparison of peak memory and latency on an MCU, i.e., NUCLEO-144 board. (d) Comparison of model size and latency on a Raspberry Pi 3. (e) Latency comparison of cutlass-int8-TC, cutlass-BNN-TC, and APNN-w1a2 for various models on an RTX 3090 GPU. (f) Performance comparison (in tokens/s) between two libraries running a 1.58-bit LLM (BitNet-3B).

read/write speed by storing multiple operands within a single 32- or 64-bit word. Second, we examine acceleration through kernel-level optimizations, showcasing how much faster BNNs/TNNs can perform inference compared to FP32 or INT8 models on various hardware platforms. Finally, we survey the specialized software libraries available for optimizing BNNs/TNNs.

A. Acceleration by Using Bit Packing

This section outlines the necessary steps for accelerating BNNs and TNNs on off-the-shelf hardware. Processing low-bit weights and activations individually is highly inefficient on modern CPUs, which handle a minimum of 32-bit data per instruction, resulting in underutilized processing power. Thus, effective acceleration hinges on techniques that allow these low-bit values to be packed in a 32- or 64-bit data so that multiple operands are processed in parallel.

Fig. 7(a) illustrates the preparation step for executing a binary convolution operation—which consists of 1-bit weights and 1-bit activations—on a 32-bit CPU architecture. First, the convolution operation is converted into a dot product form using the im2col method. Following this, 32 individual binary values are consolidated into a single 32-bit register via bit packing, facilitating their simultaneous loading for significantly faster data read compared to reading each bit individually. Fig. 7(b) presents a source code snippet that executes a dot product between bit-packed weights and activations. This code accumulates the results of 32 individual multiplications and stores the output to variable z. The 32 multiplications are done by using a single bitwise XOR instruction

on 32-bit operands `x[j]` and `w[w_idx]`, followed by the `__builtin_popcount` intrinsic function. Subsequently, after the accumulation in z is complete, the value is binarized to serve as the input for the next layer—it is converted to 1 if positive and 0 if negative. By storing this binary result directly into the output buffer (`out`), it dramatically reduces memory usage during the computation process.

B. Kernel Optimization for BNN/TNN Acceleration

This section covers kernel optimization methods that have been proposed to improve the performance of BNNs and TNNs on various hardware platforms.

1) Kernel Optimization in Resource-Constrained MCUs: CBin-NN [98] is a framework designed for BNN acceleration on MCUs. It achieves high inference speed by applying a series of targeted optimizations to essential BNN operations. These techniques include bit packing to reduce data load, loop unrolling of convolution layers to enable instruction-level parallelism, and the fusion of batch normalization and activation within convolution layers to decrease both the computation and peak memory usage. A key advantage of CBin-NN is that its entire codebase is implemented in C++, making it significantly less hardware-dependent than many existing BNN libraries [99], [100]. When the framework was applied to "SmallCifar," a custom model optimized for BNNs, it demonstrated a 15% speedup and a 13.4× reduction in peak memory consumption compared to CMSIS-NN [101], a conventional 8-bit deep learning library for MCUs, as shown in Fig. 7(c).

BitNetMCU [102] is a library designed to enable low-bit quantization on a RISC-V-based CH32V003 32-bit MCU. This low-cost MCU (under U.S. 0.1) presents significant design challenges: its architecture only supports integer arithmetic, making quantization a necessity, and its severe memory constraints—16 kB of flash and 2 kB of SRAM—demand highly compact model representations. To address these limitations, BitNetMCU provides flexible support for a wide range of weight bit-widths, including 8 bit, 4 bit, 2 bit, ternary, and binary, allowing developers to design models with optimal size, and supports 16-bit integer/INT8 activations, ensuring that all computations can be executed exclusively in the integer domain. Furthermore, future plans for BitNetMCU include enabling W4A4 (4-bit weight and 4-bit activation) computation using the NF4 [103] format via LUTs, with the goal of delivering higher performance in more general settings.

2) Acceleration on High-Performance Edge Platforms: The fast bitwise computation kernel (FBCK), introduced in the BiFSMNv2 [104] article, is a specialized kernel targeting the ARMv8 architecture. It was proposed to address a key limitation in previous work such as daBNN [105] and Bolt [99], which lacked optimized register handling for binarized general matrix multiplication (BGEMM) operations. FBCK's methodology involves storing data in registers and maximizes the data reuse throughout the computation, thereby executing BGEMM with high efficiency and minimal wasted register operations. As a result, when applied to the BiFSMNv2 model, FBCK achieves a 54.1%/14.4% faster inference speed compared to the daBNN/Bolt BNN library [Fig. 7(d)].

APNN-TC [106] is a kernel that supports arbitrary-precision computation by leveraging the native 1-bit AND and XOR operations available on Ampere GPU Tensor Cores. This article demonstrates that an operation involving n-bit weights and t-bit activations can be decomposed into $n \times t$ individual 1-bit operations. This principle enables the acceleration of nonnative bit-precision computations by utilizing the 1-bit capabilities of the Tensor Cores. For example, an APNN operation with binary weights (W) and 2-bit activations (A) is performed by first decomposing the 2-bit activation into its most significant bit (MSB), A_m, and least significant bit (LSB), A_l. Each of these components is, then, multiplied by the binary weights using a binary matrix–matrix multiplication (BMMA) operation. The final result is obtained by shifting the result from A_m and adding it to the result from A_l. This can be expressed by the following formula:

$$W \odot A = \text{BMMA}(W, A_m) \ll 1 + \text{BMMA}(W, A_b). \tag{16}$$

Following this principle, a general W^pA^q GEMM can be implemented using $p \times q$ BMMA operations combined with shift-and-add operations. Since BMMA is composed of computationally inexpensive bitwise operations, i.e., and, XOR, and popcount, this approach is significantly faster in low-bit scenarios than conventional INT8 kernels. Furthermore, this article introduces additional optimizations such as batch-based double caching, bit-packing, channel-major data organization on convolution layers, and input-aware padding to further accelerate the APNN kernel. On the VGG-variant and ResNet-18 models, as shown in Fig. 7(e), the resulting APNN-w1a2 kernel demonstrates significant speedups, outperforming the standard INT8 kernel by 15.5× and 89.2×, and a BNN kernel by 1.3× and 1.1×, respectively.

T-MAC [107] is a library targeting LLMs that utilize low-bit weight-only quantization. Conventional weight-only quantization follows an inefficient process of dequantizing low-bit weights to a higher precision, such as FP16, before performing matrix multiplication with the activations. T-MAC accelerates inference by replacing this process with a CPU-optimized LUT-based computation. The core idea of T-MAC is the bit-serial decomposition of an n-bit weight matrix, W, into n individual 1-bit matrices, W_i. With this method, the original matrix multiplication can be transformed into a sum of multiple binary matrix multiplications as follows:

$$A \odot W = A \odot \sum_{i=0}^{n-1} 2^i \cdot W_i = \sum_{i=0}^{n-1} 2^i \cdot A \odot W_i. \tag{17}$$

Since the elements of each 1-bit matrix W_i consist of only two values, i.e., −1 and 1, the operation $A \odot W_i$ is effectively reduced to additions and subtractions of the activation values. T-MAC takes this step further by grouping the activation vector A into g elements and precalculating the results for all possible 1-bit patterns (2^g combinations). These precalculated results are, then, stored in an LUT. Thus, (17) is reformulated to

$$A \odot W = \sum_{i=0}^{n-1} 2^i \cdot A \odot W_i = \sum_{k=0}^{d/g} \sum_{i=0}^{n-1} 2^i \cdot \text{LUT}_{2^g}^k \left(W_i^k\right) \tag{18}$$

where d is the input dimension, k is the group index, and $\text{LUT}_{2^g}^k$ is generated to store all possible partial sums for a given

TABLE V
COMPARISON OF LOW-BIT SOFTWARE LIBRARIES

Library	Support Type	Supported Hardware	Speed Up
QKeras [109]	BNN, BWN, TNN, TWN	x86, ARM, GPU, iOS, MCU, FPGA	No
QONNX [116]	BNN, BWN, TNN, TWN	x86, ARM, GPU, iOS, MCU, FPGA	No
CoreML [115]	BWN	iOS	Yes
Larq [100]	BNN, BWN	x86, ARM, MCU	Yes
Bolt [99]	BNN, BWN	x86, ARM, GPU, iOS, MCU	Yes
FINN [113]	BNN	FPGA	Yes
BitNet.cpp [117]	TWN	x86, ARM, GPU	Yes

group of activations. For instance, with a group size of $g = 4$, LUT_{16}^{0} contains 16 entries, representing every possible partial sum from $(A_0 + A_1 + A_2 + A_3)$ down to $(-A_0 - A_1 - A_2 - A_3)$ and LUT_{16}^{1} contains the partial sums for the next group of activations, i.e., $\{A_4, A_5, A_6, A_7\}$. While this method introduces the initial overhead of precomputing the LUT, it completely bypasses the need for dequantization. Instead, the bit pattern of the weights corresponding to an activation group is used as an index to directly retrieve the partial sum from the LUT. This technique is especially effective on resource-constrained edge devices. As shown in Fig. 7(f), on a Raspberry Pi 5, this method achieves a 5.8× higher throughput for the BitNet-3B [40] model based on ternary weights compared to the dequantization-based approach used in llama.cpp [108].

C. Library Support for BNN/TNN in Commercial Hardware

Rather than providing a comprehensive survey of all BNN/TNN frameworks, this article focuses on software libraries that have been updated since 2024. The selected libraries that support BNN/TNN/BWN/TWN are summarized in Table V. Some libraries provide speedup on commercial hardware either by using third-party backends or a native inference engine. A key observation from this article is that most of these frameworks are not dedicated exclusively to BNNs, but are rather general-purpose quantization libraries that also provide support for binary networks. Therefore, our subsequent discussion on each framework will primarily focus on its capabilities for accelerating low-bit operations.

1) Frameworks for BNN/TNN Design and Simulation: QKeras [109], a quantization library built upon TensorFlow and Keras [110], supports QAT and is notable for its user-friendly API and flexible support for various bit precisions. Beyond standard integer quantization, QAT is available for BNNs and TNNs, enabling the simulation and evaluation of accuracy for extremely low-bit models. In contrast, QONNX is a toolset based on the ONNX intermediate representation; while it does not support training, its strength lies in importing and processing models from other QAT-enabled frameworks, e.g., QKeras and Brevitas [111]. Neither QKeras NOR QONNX performs hardware acceleration directly. Instead, they serve as crucial front-end tools that enable acceleration by preparing models for backend compilers such as TFLite [112] (mobile devices and MCU), FINN [113] [field-programmable gate array (FPGA)], and HLS4ML [114] (FPGA), so that the models can be deployed on specific hardware targets.

2) Frameworks for Accelerated Low-Bit Inference: Core ML [115] is Apple's on-device inference engine and acceleration framework, optimized specifically for Apple Silicon. It supports various model optimization techniques, including standard quantization algorithms and pruning. A key feature is palletization, which is a method of clustering weights that enables them to be compressed to 1, 2, 3, 4, 6, or 8 bit. While this technique significantly reduces a model's memory footprint, the actual computations require the weights to be dequantized back to FP16 format. In other words, this process only uses FP16 computational flow. Therefore, palletization does not substantially improve overall inference speed unless a task is memory-bounded. For instance, applying 2-bit palletization to a ResNet-50 model reduces memory usage 7.63×, but improves end-to-end inference speed by only about 6%.

Larq [100] is an open source deep learning library specifically developed for the acceleration of BNNs. It is designed for the efficient deployment of BNNs on mobile or edge devices, featuring highly optimized kernels and a custom inference engine called the Larq compute engine. This focuses on specialized support for binary operations distinguishes it from more general-purpose frameworks. In particular, alongside QKeras, Larq is one of the BNN libraries officially supported for deployment by the prominent MCU manufacturer STMicroelectronics, via its X-Cube-AI development toolchain.

Bolt [99] is a lightweight inference framework developed by Huawei Noah's Ark Laboratory, designed to accelerate quantized models across a variety of hardware platforms. It specifically optimizes INT8 and binary operations by leveraging the vector instruction sets available on ARM and ×86 architectures. To improve usability and streamline the deployment process, Bolt includes a converter tool named ×2bolt. This utility allows users to port models originally developed in other popular deep learning frameworks into the Bolt format for accelerated execution.

FINN [113] is a specialized framework from Xilinx [acquired by Advanced Micro Devices (AMD)] for creating highly optimized hardware accelerators for BNNs on Xilinx FPGAs. It takes BNN models trained using libraries like Brevitas or processed through QONNX and uses the FINN compiler to transform them. This process generates customized register-transfer level (RTL) descriptions, either through high-level synthesis (HLS) or directly in Verilog, creating a hardware design that is specifically optimized for the given model on a target Xilinx FPGA board.

BitNet.cpp [117], a derivative of the llama.cpp [108] project, accelerates LLM inference for models using ternary weights. The library boosts end-to-end throughput by minimizing the memory footprint of the weights via distinct packing methods tailored for ternary operations. This approach yields a performance boost by up to 4.91× against the FP16 baseline on an Apple M2 Ultra.

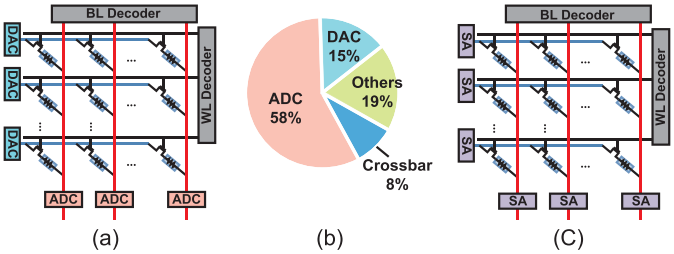


Fig. 8. Compute-in-memory architecture based on NVM. (a) nvCIM architecture. (b) nvCIM energy breakdown. (c) NVM-based BNN CIM architecture.

VI. Compute-in-Memory Technology for Binary/Ternary Neural Networks

CIM has gained significant attention as an efficient solution for artificial intelligence hardware with strict power consumption constraints. Unlike traditional von Neumann architectures, where the energy cost of data transfer between off-chip memory and the processing unit often exceeds the energy used for computation [118], CIM embeds computation directly within memory, significantly reducing the power overhead associated with data movement. This section provides an overview of CIM architectures specifically optimized for BNNs and TNNs, which employ extreme quantization. The focus is distinct from the broader category of general-purpose CIM architectures [119], [120], [121], which are designed to support conventional neural networks with multibit precision.

A. Preliminary on Compute-in-Memory for BNN/TNN

A typical CIM architecture, as illustrated in Fig. 8, is structured to perform matrix–vector multiplication (MVM) operations directly within the memory. This architecture includes a digital-to-analog converter (DAC) to transform digital input vectors into analog signals, a crossbar array of memory cells to store weights and facilitate MVM computations, and an analog-to-digital converter (ADC) to convert the analog results back into digital form. Additionally, shift and adder units are employed for the accumulation of partial sums.

In conventional deep neural networks (DNNs), CIM architectures typically convert inputs and weights into integer representations. Weights are stored in the crossbar array, while inputs are applied to the word lines (WLs) through the DAC, enabling MVM operations. The resulting columnwise analog sums are subsequently digitized by the ADC, and further operations, such as shifting and adding, are performed to finalize the computations. When processing multibit inputs, such as INT8, the computation may be carried out bit-serially over multiple cycles, or the DAC may be designed to handle multibit operations, both of which increase the energy consumption.

In contrast, BNNs and TNNs, where inputs and weights are limited to the sets $\{-1, 1\}$ or $\{-1, 0, 1\}$, provide significant advantages in terms of energy efficiency. The conversion of digital inputs into analog signals can be accomplished using a simple 1-bit DAC, substantially reducing complexity and power consumption. Moreover, as depicted in Fig. 8(b), the

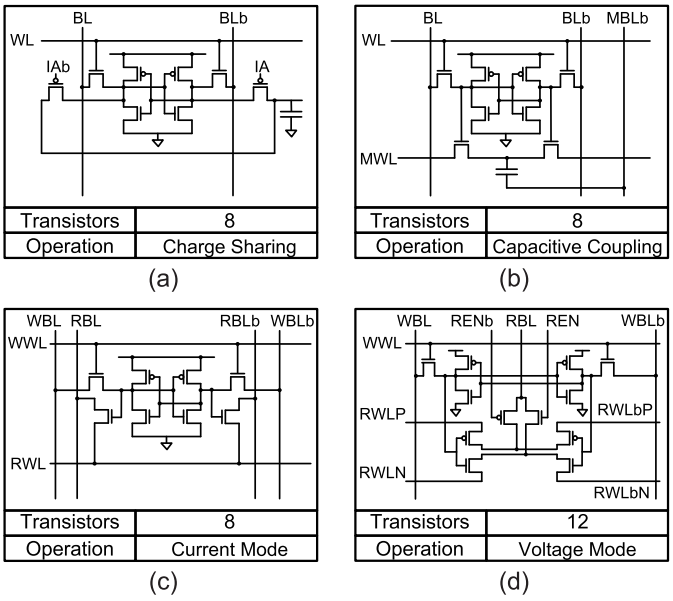


Fig. 9. SRAM-CIMbit-cell schematics. (a) Valavi et al.'s [122] charge-sharing-based 8T1C SRAM-CIM. (b) Jiang et al.'s [42] capacitive coupling-based 8T1C SRAM-CIM. (c) Oh et al.'s [12] current mode 8T SRAM-CIM. (d) Yin et al.'s [44] voltage mode 12T SRAM-CIM.

memory cells in BNN/TNN architectures need only store binary or ternary values, which simplifies the logic within each cell and minimizes the complexity of the peripheral circuitry or converted ADC to sense amplifier (SA). These considerations make CIM architectures highly suitable for BNN and TNN implementations, and this article aims to focus on such optimized architectures for achieving efficient on-device processing.

B. SRAM Compute-in-Memory for BNN/TNN

SRAM-based CIM offers several advantages, including ease of implementation using standard CMOS logic processes and its high-speed read/write operations. This approach can be realized with minimal modifications to conventional six-transistor (6T) SRAM, while also incorporating design enhancements, such as additional transistors or capacitors to reduce read/write disturbances and mitigate nonlinearity in multiply and accumulate (MAC) operations.

Analog CIM designs are classified into three main categories: current mode, voltage mode, and charge domain, which includes charge sharing and capacitive coupling, as shown in Fig. 9. In all three categories, multiplications occur within SRAM bit-cells, while accumulations take place in bit-lines (BLs). In current-mode designs, multiplication results in bit-cells that cause either charging or discharging of the shared BL. In contrast, voltage-mode designs involve each bit-cell pulling up or down the shared BL based on its result. Charge-domain designs, distinct from the other two, use capacitances to compute MAC operations and are further divided into two types: charge sharing and capacitive coupling. Charge sharing works by shorting charged or discharged capacitors together, averaging the voltage in the BL. Capacitive cou-

TABLE VI
COMPARATIVE ANALYSIS ON SRAM-COMPUTE-IN-MEMORY ARCHITECTURES FOR BNNS/TNNS

Reference	Tech. (nm)	Bit-cell	MAC mode	Input	Weight	Output bit	TOPS/W
Valavi et al. [122]	65	8T1C	**CS**	-1/1	-1/1	1	866
Si et al. [123]	65	6T	**V**	-1/1	-1/1	1	30.5
				0/1	-1/1	1	55.8
Jiang et al. [42]	65	8T1C	**CC**	-1/0/1	-1/1	3.46	671.5
Jain et al. [124]	32	19T	**V**	-1/0/1	-1/0/1	3	132.7
Yin et al. [44]	65	12T	**V**	-1/0/1	-1/0/1	3.46	403
Oh et al. [12]	28	8T2C	**CS**	0/1	-1/1	1	1607-3182
Kang et al. [45]	28	6T (B)	**V**	-1/0/1	-1/1	4	1087
		12T (T)	**V**	-1/0/1	-1/0/1	4	547.1
Yu et al. [125]	68	8T	**CR**	0/1	-1/1	1-5	15.8-490
Nguyen et al. [126]	28	10T	**CR**	-1/0/1	-1/0/1	1.58	333.8
Jeong et al. [127]	65	16T1C	**CC**	-1/0/1	-1/0/1	1	823
Cheon et al. [128]	28	10T	**CS**	-1/0/1	-1/1	3.32	942.9-2680
Shah et al. [11]	65	6T	**CR**	0/1	-1/1	1-5	157.6-984

CS is charge sharing, **CC** is capacitive coupling, **V** is voltage mode, **CR** is current mode.

pling, on the other hand, directly generates the accumulated voltage. We have categorized CIM architectures optimized for BNNs and TNNs according to the four techniques, as shown in Table VI.

1) Charge-Domain Computation of SRAM-CIM: Valavi et al. [122] improve energy efficiency by integrating two additional p-channel metal–oxide–semiconductor (PMOS) transistors within the SRAM cell to perform XNOR operations, complemented by a single metal–oxide–metal (MOM) capacitor for charge-domain accumulation. The design also leverages clock-gating to adjust activity factors at the convolution filter level, further optimizing energy consumption.

Oh et al. [12] introduced the 8-transistor/2-capacitor (8T2C) SRAM, which mimics DRAM operation by utilizing a charge-sharing mechanism within the SRAM cell. This design operates in two stages: first, an internal charge transfer where the weight stored in each cell interacts with the input; second, a BL charge-sharing stage where the computed result is propagated. By deactivating the WL when the input is zero, the design reduces power consumption, making it more energy-efficient than conventional approaches.

Cheon et al. [128] minimized area overhead by using existing junction capacitances with the addition of two transistors, eliminating the need for external capacitors such as MOM capacitors. Furthermore, the design includes an ADC that fits within a single SRAM column, reducing overall chip area.

Jiang et al. [42] enhanced SRAM-CIM by integrating two additional n-channel metal–oxide–semiconductor (NMOS) transistors into the traditional 6T-SRAM, along with dedicated WL and BL for CIM operations. An added metal–oxide–semiconductor capacitor (MOSCAP) enhances the robustness of MAC operation results. Although this increases the overall cell size compared to conventional 6T SRAM, the MOSCAP's higher capacitive density compared to a MOM capacitor improves the linearity and stability of MAC operations.

Jeong et al. [127] proposed a ternary CIM processor that significantly enhances energy efficiency by eliminating the need for an ADC. The design incorporates a compact 16-transistor/1-capacitor (16T1C) ternary bit-cell architecture that improves linearity while maintaining high processing speeds through capacitive coupling-based MAC operations. Unlike traditional approaches that require converting analog values to digital for partial summations across columns, their design employs a charge-based partial sum adder (CPA), allowing direct partial summation on analog values, further boosting energy efficiency.

2) Voltage-Mode Computation of SRAM-CIM: Si et al. [123] introduced a dual-split 6T-SRAM structure that partitions the WL into two segments: WLL and WLR. This configuration enables the SRAM to handle read, write, and CIM operations without requiring additional transistors or capacitors. The cell supports XNOR operations with inputs $\{-1, 1\}$ and BNN operations with inputs of $\{0, 1\}$, all controlled by the signals applied to the split WLs (WLL and WLR).

Jain et al. [124] introduced an SRAM cell redesigned with 19 transistors, optimized for efficient handling of ternary inputs and ternary weights. This structure supports both symmetric ternary weights $\{-1, 0, 1\}$ and asymmetric ternary weights $\{-\beta, 0, \alpha\}$, as defined in [59]. The cell and peripheral circuitry are designed to accommodate both types of ternary weights, ensuring versatile and efficient processing.

Yin et al. [44] introduced a 12-transistor (12T) bit-cell structure that integrates a 6T SRAM core with additional circuitry to perform XNOR operations. Four transistors execute the XNOR function, while two transistors control its activation. In standard memory mode, the cell operates like a conventional SRAM, supporting typical read and write functions. In XNOR mode, the cell computes XNOR operations between stored weights and input values, with the resulting voltage changes on the read BL accumulating the XNOR outputs from multiple bit-cells.

Kang et al. [45] introduced a method that counts the number of zeros in the ternary input to perform computations without significantly modifying the traditional 6T SRAM structure. In conventional 6T SRAM-CIM, computations rely on voltage differences generated by pull-up and pull-down operations, with the pull-down process often dominating, leading to reduced accuracy. To address this, the approach in [45] calculates the pull-up value based on the pull-down value and the number of zeros in the input, improving the accuracy of MAC operations compared to the traditional method.

3) Current-Mode Computation of SRAM-CIM: Yu et al. [125] designed a CIM system based on an 8T-SRAM structure, separating the BL for CIM operations from the one used for standard read/write operations, effectively preventing disturbances during read/write processes. To further optimize the design, they incorporated a column-based ADC, using additional SRAM cells within each column to operate and calibrate the ADC via an offset. This distributed ADC setup minimizes area and energy overhead by spreading the ADCs across columns, while boosting throughput through parallel processing. The ADC's reconfigurable resolution (1–5 bits) enhances adaptability and energy efficiency, and the integrated offset calibration ensures high accuracy by reducing nonlinearity and mismatch errors.

Nguyen et al. [126] developed a bit-cell utilizing ten transistors to represent ternary weights, effectively reducing area compared to earlier approaches that either combined two 6T SRAM cells or employed additional transistors for custom SRAM configurations. To tackle the high power consumption associated with traditional voltage-mode CIM operations, they also proposed a current-based SA, which significantly reduces power usage while providing ternary outputs.

Shah et al. [11] designed a CIM architecture using six PMOS transistors to reduce read disturbances. PMOS transistors offer greater resilience to read disturbances than NMOS transistors, as they can operate at a higher gate overdrive voltage while maintaining the same drive current. Furthermore, by incorporating a MOMCAP above the crossbar array, the BL capacitance is increased, enhancing the bit-cell's resistance to read disturbances without adding any area overhead.

C. Nonvolatile Compute-in-Memory for BNN/TNN

Nonvolatile memory (NVM) offers several key advantages over SRAM, including higher energy efficiency, data retention, multibit storage in a multilevel cell (MLC), and a smaller bit-cell area. Unlike SRAM, which requires continuous power to maintain data, NVM preserves information even when powered off, allowing for energy savings and instant-on functionality in low-power devices like MCUs. This makes NVM suitable for power-hungry edge-computing systems. Moreover, NVM-based CIM (nvCIM) enhances the efficiency of machine learning tasks by both storing neural network weights and executing computations directly within the memory. This approach overcomes SRAM's limitations, such as its inability to retain data long-term and its higher power requirements for continuous operation [119], [120], [121], [129], [130], [131].

Leveraging these characteristics, the development of nvCIMs optimized for the IoT applications is also actively ongoing. For instance, Kim et al. [132] maximized efficiency in a keyword spotting system by processing the computationally light depthwise convolution (DW-Conv) in the digital domain, while executing the pointwise convolution within a multibit resistive RAM (RRAM) CIM. Antolini et al. [133] showed that by applying on-chip current-induced annealing and HW/SW drift compensation, phase-change memory (PCM) arrays can sustain stable multilevel conductance over long durations, thereby enabling real-time control and predictive maintenance tasks such as motor speed regulation and vibration anomaly detection in smart factories. Quqa et al. [134] presented a PCM-based smart sensing platform for structural health monitoring, in which PCM's nonvolatility and analog programmability support low-power, in-sensor pattern extraction. In the broader IoT contexts, domain-specific analog in-memory computing (AIMC) frameworks have also been explored for binary pattern matching and event-driven sensing, further emphasizing the cross-device generality of in-memory computing for edge intelligence [135].

However, storing multibit values per cell presents two major challenges for analog CIM. First, device-level nonidealities (e.g., conductance variability, finite programming precision, and circuit-level nonlinearity) cause a significant accuracy degradation compared to the ideal software model [136], [137]. Second, multibit MVM operations produce a larger analog output range, demanding high-resolution ADCs. These high-bit ADCs are a well-known bottleneck [138], [139], [140], exponentially increasing the system's overall power consumption and area. Given these multibit issues, this section highlights those specifically optimized for BNN and TNN, as shown in Table VII.

1) RRAM-Based CIM for BNN/TNN: RRAM is a promising solution for nvCIM systems, offering a small memory cell size and fast access speeds. RRAM operates on the principle that a dielectric, typically an insulator, can become conductive by forming a filament or conduction path when a sufficiently high voltage is applied. This enables RRAM bit-cells to store data as resistance, as shown in Fig. 10(a) and (b), ranging from simple 1-bit storage with high-resistance state (HRS) and low-resistance state (LRS) to more advanced configurations capable of storing floating-point values using multilevel resistance. However, as bit precision increases, the stored data become more vulnerable to noise, which poses a significant challenge. To address this, we discuss an approach optimized for binary and ternary CIM operations, where each bit-cell stores either 1 or 1.58 bit, providing a solution tailored to low-precision computing for BNNs and TNNs.

Sun et al. [10] use two 1T1R bit-cells to represent each weight in XNOR-based MAC operations, enabling parallel processing. In this design, a weight of + 1 is encoded as (LRS, HRS) across two bit-cells, while −1 is represented as (HRS, LRS). The input is applied as voltage to the bit-cells, with + 1 encoded as (1, 0) and −1 as (0, 1), facilitating the XNOR operation. For example, when both the input and weight are −1, the first cell is in HRS and the second in LRS, resulting in a large current and an output of + 1. Conversely, if the weight is + 1, the first cell is in LRS and the second in HRS,

TABLE VII
CIM BASED ON NVM (RRAM, MRAM, AND PCRAM) FOR BNNS/TNNS

Reference	Tech. (nm)	Device	Bit-cell	Input	Weights	Output Bits	TOPS/W
Sun et al. [10]	65	RRAM	1T1R†	-1/1	-1/1	3	141.2
Yin et al. [141]	90	RRAM	1T1R†	-1/1	-1/1	3	24.1
Yoon et al. [142]	40	RRAM	1T1R	0/1	-1/0/1	4	118.4
Li et al. [9]	180	RRAM	3T2R	-1/1	-1/1	1	51.3
Fan et al. [143]	45	MRAM	2T1M	0/1	0/1	1†⋆	-
Angizi et al. [144]	45	MRAM	1T1M	0/1	0/1	1‡⋆	-
Pham et al. [145]	65	MRAM	2T2M	-1/1	-1/1	1	311
Na et al. [146]	28	MRAM	1T1M	0/1	-1/1	1‡	58.7
Zhu et al. [147]	45	MRAM	1T1M	-1/0/1	-1/0/1	1‡⋆	-
Joshi et al. [137]	90	PCRAM	-	8	-1/0/1*	8	11.9
Shahroodi et al. [148]	-	PCRAM	1T1R	-1/1	-1/1	-	-

† is divided positive bit-cell and negative bit-cell.
‡ is element-wise operation
⋆ is supported Boolean logic operation (AND, OR, etc.)
* is designed synaptic weights, can support high-bit weight (INT8, FP32).

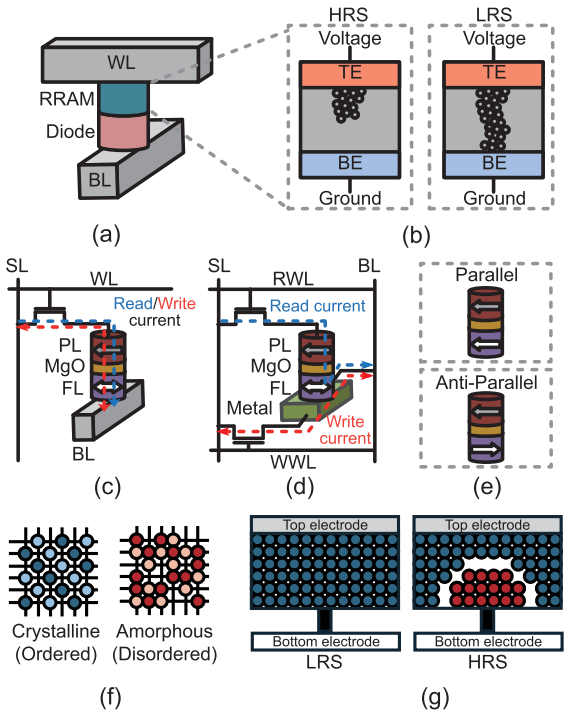


Fig. 10. RRAM and MRAM and PCRAM structures and two states for programming 0/1. (a) RRAM. (b) RRAM state. (c) STT-MRAM. (d) SOT-MRAM. (e) MRAM state. (f) Chalcogenide state. (g) PCRAM state.

producing a smaller current and an output of −1. To minimize quantization error between continuous analog and discrete digital partial sums, a current-sensing amplifier (CSA) with eight nonlinear quantization levels was implemented using the Lloyd-Max algorithm [149].

Yin et al. [141] fabricated Sun et al.'s [10] XNOR-RRAM using a 90-nm process and designed the peripheral circuits required for MAC operations. Additionally, they improved the robustness of the MAC results by replacing the conventional CSA with an ADC based on a voltage-mode SA.

Yoon et al. [142] designed a voltage-sensing multilevel 1T1C RRAM cell to prevent the logical ambiguity that occurs in conventional current-sensing RRAM. In current sensing, the accumulated current from multiple HRS cells complicates the distinction between HRS and LRS states. The voltage-sensing approach mitigates this by providing clearer differentiation between resistance states. Additionally, they designed a 4-bit ADC optimized for 3×3 ternary convolution operations. To further reduce ambiguity in the multibit RRAM cell, an iterative write pulse adjustment logic was implemented, which iteratively adjusts the write pulse based on two resistance thresholds, enabling robust representation of ternary or 2-bit values.

Li et al. [9] proposed an ADC-less CIM design to address the substantial power and area consumption associated with ADCs in conventional CIM operations. In BNNs, the binary MAC result undergoes batch normalization followed by a sign function, converting it into a binary value for the next layer. Traditional CIM systems use an ADC to generate the digital output before applying batch normalization and the sign function. In this design, batch normalization and the sign function are performed directly within the CIM, eliminating the need for an ADC and ensuring a 1-bit output. Additionally, the design reduces HRS and LRS variations, commonly seen in 1T1R and 2T2R configurations, by employing a 3T2R structure.

2) MRAM-Based CIM for BNN/TNN: Spintronic-based magnetoresistive RAM (MRAM) is a type of memristor memory that utilizes CoFe/MgO magnetic tunnel junctions (MTJs), consisting of two ferromagnetic layers separated by a tunneling barrier. The magnetization direction of one ferromagnetic layer can be switched between parallel and antiparallel states [as shown in Fig. 10(e)], representing binary data. MRAM primarily comes in two forms: spin-orbit torque-based (SOT-MRAM) [Fig. 10(d)] and spin transfer torque-based (STT-MRAM) [Fig. 10(c)], each with distinct advantages. SOT-MRAM offers faster write speeds

and enhanced endurance by using separate read and write paths, which mitigates read-disturb errors and ensure reliable performance in high-speed applications. Additionally, it requires lower switching currents, improving energy efficiency. STT-MRAM, with its simpler structure, integrates well with CMOS technology and consumes less power in standby mode, making it ideal for low-power applications. Its scalability to smaller nodes also broadens its application range. This section aims to introduce a CIM optimized for BNNs/TNNs that leverages both SOT-MRAM and STT-MRAM technologies.

Fan and Angizi [143] developed a 2T1M SOT-MRAM CIM capable of executing logical AND and OR operations on individual elements of two vectors. Beyond standard memory read operations, where a single row is activated, the circuit enables simultaneous activation of two rows, allowing and or OR operations between their elements through voltage manipulation on the BL. This study further distinguishes between computations performed within the CIM and those managed by peripheral circuits, emphasizing performance improvements at each layer, particularly in terms of processing speed.

Angizi et al. [144] proposed MRIMA, a 1T1M STT-MRAM-based in-memory accelerator designed to support Boolean logic operations. A reconfigurable SA was developed by modifying the conventional SA used for read operations, enabling rowwise execution of and, OR, and majority functions. Additional SAs were incorporated to support NAND, NOR, XOR, XNOR, and NOT operations. The design also allows operations across three rows, facilitating the execution of the majority functions and combining and and xor operations to perform 1-bit addition.

Pham et al. [145] proposed STT-BNN, a CIM architecture based on 2T2M STT-MRAM that performs XNOR operations between inputs and weights. Similar to works in [10] and [141], + 1 and −1 are represented by (parallel, antiparallel) and (antiparallel, parallel) magnetization states, respectively. The input is applied to the BL connecting two STT-MRAM cells, where + 1 is represented as (1, 0) and −1 as (0, 1). In the parallel state, the resistance is low, keeping the output voltage nearly unchanged. Conversely, in the antiparallel state, higher resistance causes a lower output voltage. The architecture is designed for XNOR-based MAC operations. For example, when both the input and weight are + 1, the input voltage is applied to the parallel MTJ, yielding a high voltage (+ 1). When the input is −1, the voltage is applied to the antiparallel MTJ, resulting in a low voltage (−1). The MAC output is computed by summing the voltages from each bitcell along the source line, with the final voltage representing the processed result. Time-based sensing and voltage-boosting techniques are utilized to ensure robust MAC calculations.

Na [146] developed a 1T1R digital STT-MRAM capable of computing ternary outputs by setting the input to 0 or 1 and the weight to −1 or 1. During the row-by-row MAC operation, if the input is 0, the corresponding row is skipped, ensuring that the final result of the operation is either 1 or −1. To accumulate the elementwise multiplication results from these operations, a 7-bit noncircular counter was designed. This counter is initialized to 0111111 and increments by 1 when the multiplication result from the SA is 1, while it decrements by 1 when the result is −1. After the MAC process is complete, if the MSB of the 7-bit counter is 1, the MAC result is set to 1; if the MSB is 0, the result is set to 0, effectively approximating a sign activation.

Zhu et al. [147] introduced iMAT, a CIM optimized for TNNs that operate with ternary inputs and weights. The iMAT employs a 2T2M STT-MRAM cell for storing ternary inputs and accumulating results, while supporting row-by-row Boolean logic operations. Unlike traditional CIMs that store weights within the CIM and apply activations to the WL or BL for computation, iMAT stores activations in memory and applies weights as inputs via the WL. This approach allows for weight reuse across multiple inputs in convolution layers, enabling faster computations by skipping operations involving zero weights. Additionally, ternary MAC results are directly written back to the CIM, minimizing data movement between the CIM and external logic.

3) PCRAM-Based CIM for BNN/TNN: PCM [phase-change RAM (PCRAM)] is a next-generation NVM that stores data by changing the physical "phase" of its core material. It primarily uses chalcogenide glass, such as germanium antimony telluride (GeSbTe), which can switch between two states when subjected to Joule heating induced by an electrical current. When the material is heated above its melting point and then rapidly cooled (quenched), it enters a disordered amorphous state, as shown in Fig. 10(f). This state impedes electron flow, resulting in an HRS. Conversely, when heated above its crystallization temperature (but below the melting point) for a set duration, the atoms arrange into an orderly crystalline state [Fig. 10(f)]. This state conducts electricity easily, creating an LRS. Fig. 10(g) illustrates how these two physical states correspond to the HRS and LRS within the PCRAM cell structure. In this section, we introduce PCRAM-based CIM specifically optimized for BNNs and TNNs.

Joshi et al. [137] proposed methodologies to address weight nonidealities in PCRAM-based CIM. First, they introduced noise-aware training, which injects Gaussian noise into the weights during the learning process to build a network resilient to the nonidealities of memristive devices. This allows the model to be designed with greater robustness against hardware noise. Moreover, to mitigate the negative impact of conductance drift (resistance degradation) that occurs in PCRAM devices over time, they proposed the adaptive batch normalization statistics update (AdaBS). This technique reupdates the statistics parameters of the pretrained batch normalization layers. Through this, they demonstrated that a ResNet32-ImageNet accuracy drop of less than 1% can be achieved even after one year.

Shahroodi et al. [148] proposed TacitMap that contrasts with conventional rowwise crossbar mappings [150] by storing each weight vector vertically along a column. Specifically, each weight vector W is stored first, followed by its complemented vector $\bar{W}$ placed directly below. During inference, the corresponding input vector I and its complement $\bar{I}$ are applied in the same manner, allowing TacitMap to concatenate $(I, \bar{I})$ and apply them across the crossbar rows simultaneously. Through this organization, the XNOR + popcount operation

can be reformulated as

$$\text{Popcount}\left(\text{XNOR}\left(I,W\right)\right) = \text{Popcount}\left(\text{AND}\left(I,W\right)\right) + \text{Popcount}\left(\text{AND}\left(\bar{I},\bar{W}\right)\right). \tag{19}$$

This formulation enables the XNOR + popcount operation to be performed in a single analog step, as the combined result $\text{AND}(I,W) + \text{AND}(\bar{I},\bar{W})$ is directly read out from the columnwise ADCs. Furthermore, Shahroodi et al. [148] extended this concept beyond electronic PCRAM (ePCRAM) by introducing *EinsteinBarrier*, an accelerator based on optical PCRAM (oPCRAM). By leveraging wavelength-division multiplexing (WDM), the oPCRAM architecture can process multiple input vectors concurrently, effectively transforming the conventional MVM capability of CIM into a matrix–matrix multiplication (MMM) operation. Consequently, this design achieves a significant latency improvement (throughput) of approximately 1205× on average compared to the baseline ePCRAM implementation.

VII. Future Research Directions

In this section, we discuss embedding CIM arrays into the IoT devices as a future research direction and introduce several previous works that present potential solutions in this area. First, we discuss the current limitations of BNNs and TNNs. Second, we highlight the challenge that a standalone CIM chip is often insufficient to support the full range of operations required for deep learning models for edge intelligence. As a promising solution to this limitation, we introduce several methods that integrate RISC-V processors with CIM arrays.

A. Challenges in Deploying BNNs/TNNs

Despite their low memory footprint, achieving high computational efficiency with BNNs/TNNs still has several significant challenges. First, accelerating BNNs/TNNs necessitates kernel optimization. Since mainstream hardware frameworks like PyTorch and TensorFlow lack native support for efficient BNN/TNN acceleration, it is essential to convert and infer models using libraries with kernels specifically optimized for low-bit quantization, such as Bolt [99] or Larq [100] in Section V-C.

Second, BNNs/TNNs are difficult to use for complex tasks such as image generation [151], autonomous driving [152], and video understanding [153]. For tasks that demand intricate operations, the significant information loss from low-bit quantization can lead to a severe degradation in accuracy. However, recent research work [154], [155], [156], [157], [158] have employed BNNs/TNNs for complex tasks by enhancing the model architecture or designing new loss functions.

Finally, there is a lack of a general benchmark for comparing the performance of BNNs/TNNs. The actual inference speed of these networks varies significantly depending on the model structure and the hardware architecture. However, most current evaluation metrics assess performance solely based on OPs (i.e., 1/64 BOPs + FLOPs) derived from CPU64, which often results in discrepancies compared to actual hardware performance. To resolve this, new hardware-aware evaluation metrics are being proposed, such as the arithmetic computation effort (ACE) metric from PokeBNN [4] and PikeLPN [159], while real-hardware and task benchmarks like BiBench [160] are under continuous development. Moreover, developing BNN/TNN benchmarks specifically designed for the IoT workloads is needed as well.

B. Scalability and Reliability of CIM Technology

The scalability of CIM architectures is strongly influenced by the underlying memory device technology. SRAM-based CIM has been extensively explored due to its compatibility with standard CMOS processes and the availability of mature design methodologies. However, its cell size remains relatively large (typically 160–200 F^2), which constrains area efficiency and density. Furthermore, SRAM cells suffer from leakage current and increasing variability at advanced nodes, limiting their suitability for ultradense and energy-efficient CIM arrays [161]. While SRAM-CIM can offer high read/write speed and low design risk, these scalability issues pose challenges for large-scale deployment.

In contrast, nvCIM architectures exhibit more favorable scaling properties. The nvCIM array can achieve much higher density and support in situ analog accumulation, which enhances compute efficiency. However, these benefits come with reliability tradeoffs. Device variability, limited endurance, thermal disturbance, and the cost of integrating nonstandard memory arrays into CMOS fabrication process remain as major hurdles [162], [163]. Despite these challenges, recent studies demonstrate progress in mitigating these reliability issues, positioning nvCIM as a promising candidate for future large-scale neuromorphic and AI accelerators. In summary, SRAM-CIM provides a low-risk and design-mature path with limited scalability, while nvCIM offers greater long-term potential but requires further breakthroughs in device reliability and fabrication maturity.

C. Limitations of CIM Array Architectures

CIM is one of the most promising accelerators for deep learning operations in low-power IoT environments. By performing computations directly within the memory array, it can significantly reduce weight-loading overhead and accelerate matrix multiplications. However, despite these advantages, several critical challenges hinder its practical implementation.

First, most CIM architectures operate on 2-D arrays, necessitating that the input data be reshaped into a compatible 2-D format. This data reshaping process requires a dedicated processing core or additional hardware block, which introduces system-level overhead. Second, CIMs are highly specialized for matrix multiplication and struggle to efficiently support other types of operations. In certain neural networks, such as BNNs and TNNs, elementwise operations can be even more computationally demanding than matrix multiplications themselves [4], [159]. Third, as the operational bit-width increases, the energy consumption and error rate of ADCs rise sharply. This often necessitates supplementary processing units to handle high-precision operations separately. Therefore, designing an architecture that resolves these issues to enable

the robust and stable use of CIM is a crucial step toward realizing effective deep learning acceleration in the real-world IoT settings. In the following subsection, we introduce several research efforts aimed at addressing these challenges.

D. Co-Design of RISC-V and CIM Architectures

RV-Flash [164] is a custom MCU that integrates an eFlash-based computing-in-memory (eFlash-CIM) accelerator with a RISC-V core, specifically targeting the tinyML task of keyword spotting. The design was motivated by the observation that in BiFSMNv2 [104], the vast majority of the computational load originates from pointwise 1-D convolutions (PW-Conv). To achieve acceleration, RV-Flash employs a heterogeneous computing strategy: the computationally intensive PW-Conv operations are offloaded to a BNN-aware eFlash-CIM, while the DW-Conv and elementwise operations are handled by the RISC-V core, which has been augmented with a custom vector multiplier. This approach yielded a significant reduction in clock cycles (latency). For the BiFSMNv2_small architecture, it achieved an 89.5% cycle reduction compared to a conventional RISC-V core. Furthermore, a network architecture co-designed for the RV-Flash has been proposed, i.e., HRepFSMN, with the cycle reduction of 92.16%.

CIMR-V [165] is a BNN-based accelerator that integrates a 10T-SRAM-based CIM unit with a RISC-V Ibex core. To accelerate CIM operations from the RISC-V core, the architecture introduces three custom instructions: `cim_convolution`, `cim_read`, and `cim_write`. Furthermore, it incorporates a pipelined max-pooling block that operates concurrently with the CIM-driven convolution, a design choice that effectively minimizes the CIM array's idle time. To drastically reduce the overhead associated with weight loading, CIMR-V employs two key techniques. The first is layer fusion, where, if sufficient capacity exists, the fused weights of two consecutive layers are preloaded onto the SRAM-CIM. The second is a weight prefetching scheme that uses a dedicated weight SRAM to store the weights for the subsequent operation, allowing them to be loaded into the main SRAM-CIM immediately after the current computation is complete. By maximizing the reuse of the SRAM-CIM through these structural and dataflow optimizations, CIMR-V achieves a peak performance of 26.21 TOPS and a power efficiency of 3707 TOPS/W.

In addition to these RISC-V and CIM architectures specialized for BNNs, there is a growing trend of research combining the two in more diverse ways. Notable examples include Acore-CIM [166], which improves the accuracy of analog-CIM by using the RISC-V core to calibrate and correct errors; Vecim [167], where the standard vector processing unit in a RISC-V core is replaced by a digital CIM; and RDCIM [168], which supports multibit digital-CIM by implementing a multiprecision adaptive accumulator and a new set of RISC-V instructions.

VIII. Conclusion

In this article, we have done a comprehensive review of extreme quantization techniques, specifically binarization and ternarization, and their potential to significantly reduce model size and simplify computing engines, making them viable solutions for deploying deep learning models on edge devices with limited resources. First, we categorized extreme quantization into four distinct classes: BWN, TWN, BNN, and TNN. Each section provided an in-depth explanation of the specific advantages and tradeoffs associated with these approaches, highlighting their potential in improving both memory and computational efficiency. By addressing the latest trends in extreme quantization, we identified the strategies that are being developed to reduce the performance gap between the original model and the quantization model. Furthermore, we conducted an in-depth review of how binary and ternary quantization techniques are applied to recent Transformer architectures, which are increasingly used for solving a wide range of problems. This article provided valuable insight into the challenges and opportunities associated with implementing extreme quantization in complex models like Transformers. Finally, we discussed the advantages of various CIM architectures optimized for BNNs and TNNs, which are suitable for edge devices. In addition to reviewing commonly used SRAM-based CIM built with standard CMOS processes, we also explored CIM architectures based on emerging memory technologies such as RRAM, SOT-/STT-MRAM, and PCRAM, broadening the range of CIM design options. To allow deep learning at the edge environment in an extremely lightweight manner, it is crucial that both software and hardware research advance in tandem. We believe that this article provides a valuable foundation for future co-design efforts, bridging the gap between these two areas.

References

[1] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[3] J. Bethge, C. Bartz, H. Yang, Y. Chen, and C. Meinel, "MeliusNet: An improved network architecture for binary neural networks," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Jan. 2021, pp. 1439–1448.

[4] Y. Zhang, Z. Zhang, and L. Lew, "PokeBNN: A binary pursuit of lightweight accuracy," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12475–12485.

[5] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 525–542.

[6] P. Zhao et al., "ARBiBench: Benchmarking adversarial robustness of binarized neural networks," 2023, *arXiv:2312.13575*.

[7] Z. He, A. S. Rakin, J. Li, C. Chakrabarti, and D. Fan, "Defending and harnessing the bit-flip based adversarial weight attack," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 14095–14103.

[8] A. S. Rakin et al., "RA-BNN: Constructing a robust & accurate binary neural network using a novel network growth mechanism to defend against BFA," in *Proc. IEEE 15th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 219–228.

[9] Y. Li et al., "An ADC-less RRAM-based computing-in-memory macro with binary CNN for efficient edge AI," in *Proc. IEEE Trans. Circuits Syst. II, Exp. Briefs (TCAS-II)*, May 2023, vol. 70, no. 6, pp. 1871–1875.

[10] X. Sun, S. Yin, X. Peng, R. Liu, J.-S. Seo, and S. Yu, "XNOR-RRAM: A scalable and parallel resistive synaptic architecture for binary neural networks," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2018, pp. 1423–1428.

[11] I. Shah, K. Sarfraz, and M. Chan, "An efficient 6TP SRAM-based CIM macro with column ADCs for binarized neural networks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 5, pp. 2509–2513, May 2024.

[12] H. Oh et al., "Energy-efficient in-memory binary neural network accelerator design based on 8T2C SRAM cell," *IEEE Solid-State Circuits Lett.*, vol. 5, pp. 70–73, 2022.

[13] C. Yuan and S. S. Agaian, "A comprehensive review of binary neural network," *Artif. Intell. Rev.*, vol. 56, no. 11, pp. 12949–13013, Nov. 2023.

[14] R. Sayed, H. Azmi, H. Shawkey, A. H. Khalil, and M. Refky, "A systematic literature review on binary neural networks," *IEEE Access*, vol. 11, pp. 27546–27578, 2023.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, Jun. 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

[16] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.

[17] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[18] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[19] M. Caron et al., "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9650–9660.

[20] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 10012–10022.

[21] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 28492–28518.

[22] V. T. Mukund, P. Wang, X. Chen, T. Chen, S. Venugopalan, and Z. Wang, "Is attention all that NeRF needs?," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[23] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 4195–4205.

[24] K.-E. Lin, L. Yen-Chen, W.-S. Lai, T.-Y. Lin, Y.-C. Shih, and R. Ramamoorthi, "Vision transformer for NeRF-based view synthesis from a single input image," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2023, pp. 806–815.

[25] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[26] H. Phan et al., "MoBiNet: A mobile binary network for image classification," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Mar. 2020, pp. 3453–3462.

[27] Z. Ding, Y. Chen, N. Li, D. Zhao, Z. Sun, and C. L. P. Chen, "BNAS: Efficient neural architecture search using broad scalable architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 5004–5018, Sep. 2022.

[28] M. Lin et al., "Rotated binary neural network," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 7474–7485.

[29] Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng, "Bi-real Net: Enhancing the performance of 1-bit CNNs with improved representational capability and advanced training algorithm," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 722–737.

[30] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, "ReActNet: Towards precise binary neural network with generalized activation functions," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*. Cham, Switzerland: Springer, Sep. 2020, pp. 143–159.

[31] Y. Shang, D. Xu, B. Duan, Z. Zong, L. Nie, and Y. Yan, "Lipschitz continuity retained binary neural network," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, May 2022, pp. 603–619.

[32] P. Chen, B. Zhuang, and C. Shen, "FATNN: Fast and accurate ternary neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5199–5208.

[33] W. Xu et al., "Improving extreme low-bit quantization with soft threshold," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 4, pp. 1549–1563, Apr. 2023.

[34] Y. Li, X. Dong, S. Q. Zhang, H. Bai, Y. Chen, and W. Wang, "RTN: Reparameterized ternary network," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, 2020, pp. 4780–4787.

[35] H. You, H. Shi, Y. Guo, and Y. Lin, "ShiftAddViT: Mixture of multiplication primitives towards efficient vision transformer," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023, pp. 33319–33337.

[36] D. Wan et al., "TBN: Convolutional neural network with ternary inputs and binary weights," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 322–339.

[37] W. Huang et al., "BiLLM: Pushing the limit of post-training quantization for LLMs," 2024, *arXiv:2402.04291*.

[38] Y. Shang, Z. Yuan, Q. Wu, and Z. Dong, "PB-LLM: Partially binarized large language models," 2023, *arXiv:2310.00034*.

[39] M. Kim et al., "Token-scaled logit distillation for ternary weight generative language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Oct. 2023, pp. 42097–42118.

[40] S. Ma et al., "The era of 1-bit LLMs: All large language models are in 1.58 bits," 2024, *arXiv:2402.17764*.

[41] L. Ma, M. Sun, and Z. Shen, "FBI-LLM: Scaling up fully binarized LLMs from scratch via autoregressive distillation," 2024, *arXiv:2407.07093*.

[42] Z. Jiang, S. Yin, J.-S. Seo, and M. Seok, "C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020.

[43] B. Zhang et al., "PIMCA: A programmable in-memory computing accelerator for energy-efficient DNN inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 5, pp. 1436–1449, May 2023.

[44] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.

[45] N. Kang, H. Kim, H. Oh, and J.-J. Kim, "TAIM: Ternary activation in-memory computing hardware with 6T SRAM array," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2022, pp. 1081–1086.

[46] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.

[47] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A review of YOLO algorithm developments," *Proc. Comput. Sci.*, vol. 199, pp. 1066–1073, Jul. 2022.

[48] S. Choi et al., "Temporal convolution for real-time keyword spotting on mobile devices," 2019, *arXiv:1904.03814*.

[49] U. B. Baloğlu, M. Talo, Ö. Yıldırım, R. S. Tan, and U. R. Acharya, "Classification of myocardial infarction with multi-lead ECG signals and deep CNN," *Pattern Recognit. Lett.*, vol. 122, pp. 23–30, Jun. 2019.

[50] S. Zhang, M. Lei, Z. Yan, and L. Dai, "Deep-FSMN for large vocabulary continuous speech recognition," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2018, pp. 5869–5873.

[51] M. Courbariaux, Y. Bengio, and J. David, "BinaryConnect: Training deep neural networks with binary weights during propagations," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 3123–3131.

[52] X. Lin, C. Zhao, and W. Pan, "Towards accurate binary convolutional neural network," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 344–352.

[53] P. Yin, S. Zhang, J. Lyu, S. Osher, Y. Qi, and J. Xin, "BinaryRelax: A relaxation approach for training deep neural networks with quantized weights," *SIAM J. Imag. Sci.*, vol. 11, no. 4, pp. 2205–2223, Jan. 2018.

[54] O. Shayer, D. Levi, and E. Fetaya, "Learning discrete weights using the local reparameterization trick," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2017.

[55] A. Zhou, A. Yao, K. Wang, and Y. Chen, "Explicit loss-error-aware quantization for low-bit deep neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9426–9435.

[56] D. Lee, S. J. Kwon, B. Kim, Y. Jeon, B. Park, and J. Yun, "FleXOR: Trainable fractional quantization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1311–1321.

[57] Y. Wang, Y. Yang, F. Sun, and A. Yao, "Sub-bit neural networks: Learning to compress and accelerate binary neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5340–5349.

[58] F. Li, B. Liu, X. Wang, B. Zhang, and J. Yan, "Ternary weight networks," 2016, *arXiv:1605.04711*.

[59] C. Zhu, S. Han, H. Mao, and W. J. Dally, "Trained ternary quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Mar. 2017.

[60] Z. He, B. Gong, and D. Fan, "Optimize deep convolutional neural network with ternarized weights and high accuracy," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2019, pp. 913–921.

[61] X. Deng and Z. Zhang, "Sparsity-control ternary weight networks," *Neural Netw.*, vol. 145, pp. 221–232, Jan. 2022.

[62] C. Kyrkou, "Toward efficient convolutional neural networks with structured ternary patterns," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 5810–5817, Mar. 2025.

[63] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 29, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds., 2016, pp. 4107–4115.

[64] A. Bulat and G. Tzimiropoulos, "XNOR-Net++: Improved binary neural networks," in *Proc. Brit. Mach. Vis. Conf. (BMVC)*, 2019, p. 62.

[65] S. Kuhar, Y. Jain, A. Tumanov, and S. K. Gonugondla, "Signed binarization: Unlocking efficiency through repetition-sparsity trade-off," 2023, *arXiv:2312.01581*.

[66] Z. Tu, X. Chen, P. Ren, and Y. Wang, "AdaBin: Improving binary neural networks with adaptive binary sets," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 379–395.

[67] C. Lee, H. Kim, E. Park, and J.-J. Kim, "INSTA-BNN: Binary neural network with instance-aware threshold," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 17279–17288.

[68] M. Yuan and S. Pei, "RAD-BNN: Regulating activation distribution for accurate binary neural network," *Image Vis. Comput.*, vol. 148, Aug. 2024, Art. no. 105114.

[69] S. Xu et al., "Resilient binary neural network," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 37, 2023, pp. 10620–10628.

[70] Y. Lin, L. Niu, Y. Xiao, and R. Zhou, "Diluted binary neural network," *Pattern Recognit.*, vol. 140, Aug. 2023, Art. no. 109556.

[71] J. Xiang, Z. Chen, S. Li, Q. Wu, and Y. Liu, "OvSW: Overcoming silent weights for accurate binary neural networks," 2024, *arXiv:2407.05257*.

[72] H. Alemdar, V. Leroy, A. Prost-Boucle, and F. Pétrot, "Ternary neural networks for resource-efficient AI applications," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 2547–2554.

[73] L. Deng, P. Jiao, J. Pei, Z. Wu, and G. Li, "GXNOR-Net: Training deep neural networks with ternary weights and activations without full-precision memory under a unified discretization framework," *Neural Netw.*, vol. 100, pp. 49–58, Apr. 2018.

[74] Y. Li, W. Ding, C. Liu, B. Zhang, and G. Guo, "TRQ: Ternary neural networks with residual quantization," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 35, 2021, pp. 8538–8546.

[75] W. Xu, X. He, T. Zhao, Q. Hu, P. Wang, and J. Cheng, "Soft threshold ternary networks," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, Jul. 2020, pp. 2298–2304.

[76] S. Falkena, H. Jamali-Rad, and J. van Gemert, "LAB: Learnable activation binarizer for binary neural networks," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2023, pp. 6414–6423.

[77] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2025, pp. 5998–6008.

[78] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 10347–10357.

[79] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 34892–34916.

[80] L. Beyer et al., "PaliGemma: A versatile 3B VLM for transfer," 2024, *arXiv:2407.07726*.

[81] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "FQ-ViT: Post-training quantization for fully quantized vision transformer," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 1173–1179.

[82] Z. Li, J. Xiao, L. Yang, and Q. Gu, "RepQ-ViT: Scale reparameterization for post-training quantization of vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 17181–17190.

[83] L. Hou and J. T. Kwok, "Loss-aware weight quantization of deep networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[84] W. Zhang et al., "TernaryBERT: Distillation-aware ultra-low bit BERT," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 509–521.

[85] H. Bai et al., "BinaryBERT: Pushing the limit of BERT quantization," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, 2021, pp. 4334–4348.

[86] C. Le et al., "Efficient two-stage progressive quantization of BERT," in *Proc. 3rd Workshop Simple Efficient Natural Lang. Process. (SustaiNLP)*, 2022, pp. 1–9.

[87] H. Qin et al., "BiBERT: Accurate fully binarized BERT," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[88] Z. Liu et al., "BiT: Robustly binarized multi-distilled transformer," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 14303–14316.

[89] X. Xing et al., "BiPFT: Binary pre-trained foundation transformer with low-rank estimation of binarization residual polynomials," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2023, pp. 16094–16102.

[90] S. Xu et al., "TerViT: An efficient ternary vision transformer," 2022, *arXiv:2201.08050*.

[91] Y. Li et al., "Bi-ViT: Pushing the limit of vision transformer quantization," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 38, 2024, pp. 3243–3251.

[92] Y. He et al., "BiViT: Extremely compressed binary vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 5628–5640.

[93] P. Yin, X. Zhu, J. Song, L. Gao, and H. T. Shen, "SI-BiViT: Binarizing vision transformers with spatial interaction," in *Proc. 32nd ACM Int. Conf. Multimedia*, Oct. 2024, pp. 8169–8178.

[94] J. Xiao, Z. Li, L. Yang, and Q. Gu, "BinaryViT: Towards efficient and accurate binary vision transformers," *Proc. IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 1, pp. 195–206, Jan. 2023.

[95] Z. Wang, H. Luo, X. Xie, F. Wang, and G. Shi, "BVT-IMA: Binary vision transformer with information-modified attention," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 38, 2024, pp. 15761–15769.

[96] T. Gao et al., "BHViT: Binarized hybrid vision transformer," 2025, *arXiv:2503.02394*.

[97] W. Yu et al., "MetaFormer baselines for vision," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 2, pp. 896–912, Feb. 2024.

[98] F. Sakr et al., "CBin-NN: An inference engine for binarized neural networks," *Electronics*, vol. 13, no. 9, p. 1624, Apr. 2024.

[99] Huawei Noah's Ark Lab. (2020). *Bolt*. Accessed: Aug. 24, 2025. [Online]. Available: https://github.com/huawei-noah/bolt

[100] T. Bannink et al., "Larq compute engine: Design, benchmark, and deploy state-of-the-art binarized neural networks," in *Proc. Mach. Learn. Syst. (MLSys)*, 2021, pp. 680–695.

[101] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for arm cortex-M CPUs," 2018, *arXiv:1801.06601*.

[102] (2024). *BitNetMCU: High Accuracy Low-Bit Quantized Neural Networks on a Low-End Microcontroller*. Accessed: Aug. 24, 2025. [Online]. Available: https://github.com/cpldcpu/BitNetMCU

[103] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023, pp. 10088–10115.

[104] H. Qin et al., "BiFSMNv2: Pushing binary neural networks for keyword spotting to real-network performance," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 10674–10686, Aug. 2024.

[105] J. Zhang, Y. Pan, T. Yao, H. Zhao, and T. Mei, "DaBNN: A super fast inference framework for binary neural networks on ARM devices," in *Proc. 27th ACM Int. Conf. Multimedia*, Oct. 2019, pp. 2272–2275.

[106] B. Feng, Y. Wang, T. Geng, A. Li, and Y. Ding, "APNN-TC: Accelerating arbitrary precision neural networks on ampere GPU tensor cores," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2021, pp. 1–13.

[107] J. Wei et al., "T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge," in *Proc. 20th Eur. Conf. Comput. Syst.*, Mar. 2025, pp. 278–292.

[108] (2025). *Llama.Cpp: LLM Inference in C/C++*. Accessed: Aug. 24, 2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[109] (2020). *QKeras*. Accessed: Aug. 24, 2025. [Online]. Available: https://github.com/google/qkeras

[110] N. K. Manaswi, "Understanding and working with keras," in *Deep learning With Applications Using Python: Chatbots and Face, Object, and Speech Recognition With Tensorflow and Keras*. Cham, Switzerland: Springer, 2018, pp. 31–43.

[111] G. Franco, A. Pappalardo, and N. J. Fraser, "Xilinx/brevitas," San Jose, CA, USA, 2025, doi: 10.5281/zenodo.3333552.

[112] R. David, "Tensorflow lite micro: Embedded machine learning for tinyml systems," in *Proc. Mach. Learn. Syst.*, vol. 3, 2021, pp. 800–811.

[113] Y. Umuroglu et al., "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2017, pp. 65–74.

[114] J. Ngadiuba et al., "Compressing deep neural networks on FPGAs to binary and ternary precision with hls4ml," *Mach. Learn., Sci. Technol.*, vol. 2, no. 1, Dec. 2020, Art. no. 015001.

[115] (2017). *Core ML*. Accessed: Aug. 24, 2025. [Online]. Available: https://developer.apple.com/documentation/coreml

[116] A. Pappalardo et al., "QONNX: Representing arbitrary-precision quantized neural networks," 2022, *arXiv:2206.07527*.

[117] J. Wang et al., "Bitnet.Cpp: Efficient edge inference for ternary LLMs," 2025, *arXiv:2502.11880*.

[118] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[119] A. Shafiee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 14–26, 2016.

[120] A. Ankit et al., "PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2019, pp. 715–731.

[121] A. Lu, X. Peng, W. Li, H. Jiang, and S. Yu, "NeuroSim simulator for compute-in-memory hardware accelerator: Validation and benchmark," *Frontiers Artif. Intell.*, vol. 4, Jun. 2021, Art. no. 659060.

[122] H. Valavi, P. J. Ramadge, E. Nestler, and N. Verma, "A 64-tile 2.4-mb in-memory-computing CNN accelerator employing charge-domain compute," *IEEE J. Solid-State Circuits*, vol. 54, no. 6, pp. 1789–1799, Jun. 2019.

[123] X. Si et al., "A dual-split 6T SRAM-based computing-in-memory unit-macro with fully parallel product-sum operation for binarized DNN edge processors," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 11, pp. 4172–4185, Nov. 2019.

[124] S. Jain, S. K. Gupta, and A. Raghunathan, "TiM-DNN: Ternary in-memory accelerator for deep neural networks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1567–1577, Jul. 2020.

[125] C. Yu, T. Yoo, K. T. C. Chai, T. T.-H. Kim, and B. Kim, "A 65-nm 8T SRAM compute-in-memory macro with column ADCs for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 57, no. 11, pp. 3466–3476, Nov. 2022.

[126] T.-D. Nguyen, M.-S. Le, T.-N. Pham, and I.-J. Chang, "TRIO: A novel 10T ternary SRAM cell for area-efficient in-memory computing of ternary neural networks," in *Proc. IEEE 5th Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Jun. 2023, pp. 1–5.

[127] H. Jeong, S. Kim, K. Park, J. Jung, and K. J. Lee, "A ternary neural network computing-in-memory processor with 16T1C bitcell architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 5, pp. 1739–1743, May 2023.

[128] S. Cheon, K. Lee, and J. Park, "A 2941-TOPS/W charge-domain 10T SRAM compute-in-memory for ternary neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 2085–2097, Feb. 2023.

[129] A. Yusuf, T. Adegbija, and D. Gajaria, "Domain-specific STT-MRAM-based in-memory computing: A survey," *IEEE Access*, vol. 12, pp. 28036–28056, 2024.

[130] F. Zahoor et al., "An overview of critical applications of resistive random access memory," *Nanosc. Adv.*, vol. 6, no. 20, pp. 4980–5006, 2024.

[131] W. Sun et al., "A survey of computing-in-memory processor: From circuit to application," *IEEE Open J. Solid-State Circuits Soc.*, vol. 4, pp. 25–42, 2024.

[132] D. Kim, H. Jeong, S. Kim, H. Yoon, and K. J. Lee, "A multibit ReRAM computing-in-memory processor with adaptive decision level nonlinear ADC for ultra-low-energy keyword spotting in mobile devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, early access, Sep. 3, 2025, doi: 10.1109/TCSI.2025.3603076.

[133] A. Antolini et al., "Controlled acceleration of PCM cells time drift through on-chip current-induced annealing for AIMC multilevel MVM computation," *IEEE Trans. Electron Devices*, vol. 72, no. 1, pp. 215–221, Jan. 2025.

[134] S. Quqa et al., "Phase change memories in smart sensing solutions for structural health monitoring," *J. Comput. Civil Eng.*, vol. 36, no. 4, Jul. 2022, Art. no. 04022013.

[135] A. Antolini et al., "Combined HW/SW drift and variability mitigation for PCM-based analog in-memory computing for neural network applications," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 395–407, Mar. 2023.

[136] W. Wan et al., "A compute-in-memory chip based on resistive random-access memory," *Nature*, vol. 608, no. 7923, pp. 504–512, Aug. 2022.

[137] V. Joshi et al., "Accurate deep neural network inference using computational phase-change memory," *Nature Commun.*, vol. 11, no. 1, p. 2473, May 2020.

[138] T. Andrulis, J. S. Emer, and V. Sze, "RAELLA: Reforming the arithmetic for efficient, low-resolution, and low-loss analog PIM: No retraining required!," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–16.

[139] U. Saxena and K. Roy, "Partial-sum quantization for near ADC-less compute-in-memory accelerators," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, Aug. 2023, pp. 1–6.

[140] J. Kim, K. E. Jeon, Y. Kim, and J. H. Ko, "Column-wise quantization of weights and partial sums for accurate and efficient compute-in-memory accelerators," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.

[141] S. Yin, X. Sun, S. Yu, and J.-S. Seo, "High-throughput in-memory computing for binary deep neural networks with monolithically integrated RRAM and 90-nm CMOS," *IEEE Trans. Electron Devices*, vol. 67, no. 10, pp. 4185–4192, Oct. 2020.

[142] J. Yoon, M. Chang, W.-S. Khwa, Y.-D. Chih, M. Chang, and A. Raychowdhury, "A 40-nm 118.44-TOPS/W voltage-sensing compute-in-memory RRAM macro with write verification and multi-bit encoding," *IEEE J. Solid-State Circuits (JSSC)*, vol. 57, no. 3, pp. 845–857, Mar. 2022.

[143] D. Fan and S. Angizi, "Energy efficient in-memory binary deep neural network accelerator with dual-mode SOT-MRAM," in *Proc. IEEE Int. Conf. Comput. Design (ICCD)*, Nov. 2017, pp. 609–612.

[144] S. Angizi, Z. He, A. Awad, and D. Fan, "MRIMA: An MRAM-based in-memory accelerator," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 5, pp. 1123–1136, May 2020.

[145] T.-N. Pham, Q.-K. Trinh, I.-J. Chang, and M. Alioto, "STT-BNN: A novel STT-MRAM in-memory computing macro for binary neural networks," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 2, pp. 569–579, Jun. 2022.

[146] T. Na, "Ternary output binary neural network with zero-skipping for MRAM-based digital in-memory computing," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 7, pp. 2655–2659, Jul. 2023.

[147] S. Zhu, S. Huai, G. Xiong, and W. Liu, "IMAT: Energy-efficient in-memory acceleration for ternary neural networks with sparse dot product," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, Aug. 2023, pp. 1–6.

[148] T. Shahroodi, R. Cardoso, S. Wong, A. Bosio, I. O'Connor, and S. Hamdioui, "High-performance data mapping for BNNs on PCM-based integrated photonics," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.

[149] J. Max, "Quantizing for minimum distortion," *IEEE Trans. Inf. Theory*, vol. IT-6, no. 1, pp. 7–12, Mar. 1960.

[150] T. Hirtzlin et al., "Digital biologically plausible implementation of binarized neural networks with differential hafnium oxide resistive memory arrays," *Frontiers Neurosci.*, vol. 13, p. 1383, Jan. 2020.

[151] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NIPS*, vol. 33. Vancouver, BC, Canada: Curran Associates, 2020, pp. 6840–6851.

[152] X. Kong, T. Braunl, M. Fahmi, and Y. Wang, "A superalignment framework in autonomous driving with large language models," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2024, pp. 1715–1720.

[153] S. Buch, C. Eyzaguirre, A. Gaidon, J. Wu, L. Fei-Fei, and J. C. Niebles, "Revisiting the 'video' in video-language understanding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 2917–2927.

[154] H. Qin et al., "BiMatting: Efficient video matting via binarization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 43307–43321.

[155] K. Yin, T. Gao, and H. Kong, "Pathfinder for low-altitude aircraft with binary neural network," 2024, *arXiv:2409.08824*.

[156] K. Yin, Z. Zhang, S. Kong, T. Gao, C. Xu, and H. Kong, "Information-bottleneck driven binary neural network for change detection," 2025, *arXiv:2507.03504*.

[157] R. Wei, Z. Liu, Y. Fan, R. Wang, R. Huang, and M. Li, "SCALES: Boost binary neural network for image super-resolution with efficient scalings," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.

[158] J. Xin, N. Wang, X. Jiang, J. Li, X. Wang, and X. Gao, "Rectified binary network for single-image super-resolution," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, pp. 9341–9355, 2024.

[159] M. Neseem et al., "PikeLPN: Mitigating overlooked inefficiencies of low-precision neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 15996–16005.

[160] H. Qin et al., "BiBench: Benchmarking and analyzing network binarization," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 28351–28388.

[161] W. Haensch et al., "Compute in-memory with non-volatile elements for neural networks: A review from a co-design perspective," *Adv. Mater.*, vol. 35, no. 37, Sep. 2023, Art. no. 2204944.

[162] W.-H. Chen et al., "CMOS-integrated memristive non-volatile computing-in-memory for AI edge processors," *Nature Electron.*, vol. 2, no. 9, pp. 420–428, Aug. 2019.

[163] J.-M. Hung, C.-J. Jhang, P.-C. Wu, Y.-C. Chiu, and M.-F. Chang, "Challenges and trends of nonvolatile in-memory-computation circuits for AI edge devices," *IEEE Open J. Solid-State Circuits Soc.*, vol. 1, pp. 171–183, 2021.
[164] G. Kang et al., "RISC-V driven orchestration of vector processing units and eFlash computer-in-memory arrays for fast and accurate keyword spotting," in *Proc. 30th Asia South Pacific Design Autom. Conf.* New York, NY, USA: ACM, Jan. 2025, pp. 1174–1180.
[165] Y.-C. Guo et al., "CIMR-V: An end-to-end SRAM-based CIM accelerator with RISC-V for AI edge device," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.
[166] O. Numan et al., "Acore-CIM: Build accurate and reliable mixed-signal CIM cores with RISC-V controlled self-calibration," 2025, *arXiv:2506.15440*.
[167] Y. Wang, M. Yang, C.-P. Lo, and J. P. Kulkarni, "30.6 vecim: A 289.13 GOPS/W RISC-V vector co-processor with compute-in-memory vector register file for efficient high-performance computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 492–494.
[168] W. Yi et al., "RDCIM: RISC-V supported full-digital computing-in-memory processor with high energy efficiency and low area overhead," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1719–1732, Apr. 2024.



Dahoon Park received the B.S. and M.S. degrees in computer science from Hongik University, Seoul, Republic of Korea, in 2019 and 2021, respectively. He is currently pursuing the Ph.D. degree in electrical engineering with Korea University, Seoul.

His current research interests include algorithm-hardware co-design for energy-efficient deep learning with TinyML implementations and lightweight deep neural network.



Hyungdong Park received the B.S. degree in computer engineering from Hongik University, Seoul, Republic of Korea, in 2024, where he is currently pursuing the M.S. degree in computer engineering.

His current research interests include binary neural networks, model compression, and efficient deep learning for resource-constrained environments.



Inguk Yeo received the B.S. degree in computer engineering from Hongik University, Seoul, Republic of Korea, in 2024, where he is currently pursuing the M.S. degree in computer engineering.

His research interests include BNNs and model compression techniques, with a focus on minimizing computational overhead and enhancing inference performance in resource-efficient models.



Jiyun Kim received the B.S. degree in science and technology software from Hongik University, Sejong, Republic of Korea, in 2023, and the M.S. degree in computer engineering from Hongik University, Seoul, Republic of Korea, in 2025.

Her research interests include the development of efficient AI models through model compression and lightweight deep learning techniques.



Sunghyun Lee received the B.S. degree in electrical engineering from Hongik University, Seoul, Republic of Korea, in 2025, where he is currently pursuing the M.S. degree in computer engineering.

His current research interests include image segmentation and deep learning model compression techniques.



Suhak Lee received the B.S. degree in computer engineering from Hongik University, Seoul, Republic of Korea, in 2025, where he is currently pursuing the M.S. degree in computer engineering.

His research interests include model compression techniques, segmentation, and efficient deep learning for resource-constrained environments.



Hyunseob Shin (Student Member, IEEE) received the B.S. degree in electrical and electronic engineering from Korea University, Seoul, South Korea, in 2024, where he is currently pursuing the M.S. degree in electrical engineering.

His current research interests include compute-in-memory architectures based on eDRAM and privacy-preserving machine learning.



Sung-il Pae (Member, IEEE) received the B.S. degree in mathematics from Seoul National University, Seoul, South Korea, in 1993, and the master's degree in mathematics and the Ph.D. degree in computer science from the University of Illinois at Urbana–Champaign, Champaign, IL, USA, in 1997 and 2005, respectively.

From 2005 to 2007, he was a Research Fellow with Korea Institute for Advanced Study, Seoul. Since 2007, he has been teaching with Hongik University, Seoul, where he is an Associate Professor of computer engineering.



Deliang Fan (Senior Member, IEEE) received the M.S. and Ph.D. degrees in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2012 and 2015, respectively.

He is currently an Associate Professor with the School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ, USA. His primary research interests include machine learning circuits and algorithms at edge, in-memory computing circuits and architecture, adversarial, and trustworthy AI system. He has authored or co-authored more than 180 peer-reviewed international journal articles/conference papers in those areas.

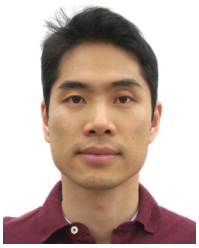
Dr. Fan was a recipient of NSF Career Award, Best IP Paper Award of DATE 2022, and Best Paper Award of GLSVLSI 2019, ISVLSI 2018, and ISVLSI 2017. His research works were also nominated as the best paper candidate of ASPDAC 2019 and ISQED 2019. He was the Program Chair of ISQED from 2023 to 2025 and the Co-Chair of iMACAW@DAC from 2022 to 2025. He was/is also the Technical Area Chair of DAC in 2021 and 2025, GLSVLSI from 2019 to 2022, and ISQED from 2019 to 2022, and the Financial Chair of ISVLSI in 2019. More details are on his research website: https://faculty.engineering.asu.edu/dfan/



Jaeha Kung (Senior Member, IEEE) received the B.S. degree in electrical engineering from Korea University, Seoul, South Korea, in 2010, the M.S. degree in electrical engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejon, South Korea, in 2012, and the Ph.D. degree in electrical and computer engineering from Georgia Institute of Technology, Atlanta, GA, USA, in 2017.

From 2017 to 2022, he was with the Department of Electrical Engineering and Computer Science, Daegu Gyeongbuk Institute of Science and Technology (DGIST), Daegu, South Korea. He is currently an Associate Professor with the School of Electrical Engineering, Korea University. His current research interests include energy-efficient digital accelerators for deep learning, distributed learning systems, hardware design for artificial intelligence, and low-power VLSI design.

Dr. Kung has served on the Technical Committee for IEEE/ACM DAC, IEEE/ACM ISLPED, IEEE ISCAS, IEEE AICAS, and IEEE HPCA.



Kon-Woo Kwon (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Korea University, Seoul, South Korea, in 2007 and 2009, respectively, and the Ph.D. degree in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2015.

He was a Research Engineer with Hyundai Motor Company, Seoul, from 2009 to 2010. He was a Component Design Engineer with Intel Corporation, Hillsboro, OR, USA, from 2015 to 2018. He is currently an Associate Professor with the Department of Computer Engineering, Hongik University, Seoul. His research interests include low-power circuits and scalable architecture design.